

对话：AI团队分工逻辑

用户

我在学习Openclaw，打算配置多个Agent也就是大龙虾团队，我是一个在线教育的创业公司，我想搭建一个完整的长期可用的AI Agent团队，我的困扰是如何分工和划分角色。

是按能力，还是岗位，还是只分名字，背后的逻辑是什么？目前业内的最佳实践是什么？给我一个完整的调研和分析

用户

我在学习Openclaw，打算配置多个Agent也就是大龙虾团队，我是一个在线教育的创业公司，我想搭建一个完整的长期可用的AI Agent团队，我的困扰是如何分工和划分角色。

是按能力，还是岗位，还是只分名字，背后的逻辑是什么？目前业内的最佳实践是什么？给我一个完整的调研和分析

助手

下面我分两部分来回答你：

1. 先给结论：业内在「多 Agent 分工」上的主流思路和底层逻辑
2. 再给你一个面向「在线教育创业公司」的可落地设计模板（含角色拆分示例 + 提示词/配置建议）

我会基于目前能查到的公开资料（比如 OpenClaw 的文档/社区文章、多 Agent 框架如 OpenAI Agents SDK、CrewAI、AutoGen 等的实践文章）来梳理；哪一部分是经验总结、哪一部分是推断，我也会说明。

一、到底应该怎么给 Agent 分工？先说逻辑，再说实践

1. 是按能力、岗位，还是名字？

从现在各大 Agent 框架和实践来看（OpenClaw、CrewAI、OpenAI Agents SDK 等），**主流不是按“名字”分，而是按「职责/能力 + 流程角色」分。**

名字只是人类好记的标签，不是核心。

更精确一点：

- 底层划分依据是：
 - （1）**任务在业务流程中的角色**（决策/执行/评审/协调）
 - （2）**所需的外部能力/工具**（访问哪类数据、调用什么 API、操作什么系统）
- 岗位和部门名称，只是把这两点映射成人类熟悉的“岗位称谓”（比如「课程运营」「教研」「班主任/学习顾问」）。

所以：

- 不是简单按“岗位名称”划分；
- 也不是按模型能力（写代码的一个、写文案的一个）那么粗；
- 而是按「在具体业务流程中承担的责任 + 能访问的工具/资源」来划。

这点在 CrewAI 的官方示例和 OpenAI Agents SDK 多 Agent 工作流示例中都很清楚：他们通常有：

- coordinator / supervisor agent（协调或路由）
- specialist agents（各类具备特定工具和上下文的“专家”）
- 有时还有 reviewer / critic agent（做质量控制）

这些本质上都是「角色 + 能力」的组合。

2. 为什么不能只按“能力”或“岗位”来切？

如果你只按一个维度切，很容易踩坑：

按“能力”切的问题（例如：文案 Agent、代码 Agent、搜索 Agent）：

- 优点：工具/权限好管理，容易重用
- 问题：
 - 对我们这种「围绕业务目标」的场景（比如从获客到转化到续费）不自然，
 - 容易出现“谁负责最终结果？”的模糊——每一步都有人做，但没人对业务 KPI 负责。
 - OpenAI Swarm 和 AutoGen 的案例里，都强调需要有「task orchestrator」来保证结果质量，否则多 Agent 只是“多人写提纲”。

按“岗位”切的问题（例如：教研 Agent、运营 Agent、销售 Agent）：

- 优点：
 - 对你这个在线教育公司很符合直觉，便于和现有团队协作
- 问题：
 - 岗位概念太大，Agent 的职责会被写得很宽泛，提示词不清晰，容易“万能但不精”，
 - 工具权限容易越界（运营 Agent 去写教学脚本，教研 Agent 去搞广告投放）。

业内比较成熟的做法是：

在业务流程上划分清晰的「阶段/任务」，
每个阶段下有一个「对结果负责的角色（owner agent）」
这个角色再调用若干「能力型子 agent / 工具」，而不是随便大家互相调用。

也就是说，「岗位」更多对应的是“对某个业务结果负责的 agent（owner）”，
「能力」则更多被抽象成：

- 子 agent（有自己提示词和工具），或者

- 该 owner agent 的可调用工具 (skills)。

3. 什么是“业内最佳实践”？我帮你抽象成 4 个设计原则

综合 OpenClaw 生态文章（例如 CSDN 的《OpenClaw/Clawdbot 实战》）、CrewAI 官方多 Agent 指南、OpenAI Agents SDK 的示例，可以抽象出四个共识：

1. 单一职责原则 (Single Responsibility)

- 每个 Agent 应该有一个明确的「成功标准」：
 - 比如“给出 3 个符合品牌调性的课程海报文案草稿”，
 - 或者“把用户过往学习记录写成一个 5 条以内的进步反馈”。
- 不要让一个 Agent 负责「从市场调研到落地执行到复盘」整条链路，否则就是“大而化之的 ChatGPT”。

2. 角色清晰 + 权限边界

- 任何一个 Agent 都要回答三个问题：
 1. 我为谁服务？（上游是谁，下游是谁/什么系统）
 2. 我能做什么？（工具权限、可访问的数据域）
 3. 我不做什么？（刻意写出来，避免越界）
- 多 Agent 框架（OpenClaw、CrewAI）里强调的「tools 限定 + scoped context」就是这个思想。

3. 有“协调者”，不是群聊

- 多 Agent 系统不是“群聊头脑风暴”，而是一个「有主持人的会议」。
- 协调者 Agent（可以叫 orchestrator / PM / lead agent）负责：
 - 把用户需求拆解成子任务
 - 决定什么时候叫哪个子 Agent
 - 收集结果并做最终汇总、解释、向用户回报
- 这是 OpenAI Swarm、OpenAI Agents SDK、CrewAI 都反复强调的模式。

4. 持续可用 = 记忆 + 可观测 + 人类兜底

- “长期可用”不是创建很多 Agent，而是：
 - 有稳定的“长期记忆”（知识库 + 用户历史 + 项目状态）
 - 有可观测性（日志、对话回放、关键决策的记录）
 - 有必要的人类监督/审核位（尤其是对外内容和商业决策）
- 多 Agent 中常见的 reviewer/critic agent，在你的场景里可以是「品牌合规审稿 Agent」「教务质检 Agent」，或者干脆是人类。

二、针对你这个「在线教育公司」，怎么设计“大龙虾团队”？

下面我给一个可以直接落地到 OpenClaw 的多 Agent 角色设计蓝图。

风格会偏实用：每个 Agent 写清楚职责、输入输出、推荐工具/能力、适合用什么提示结构。

0. 先确认你的业务链路（教育公司通用）

典型在线教育创业公司，大致有这几条主线（如果你的模式不同，后面我们可以再细化）：

1. **内容与产品线**：课程设计、教案制作、练习题与测评、教辅资料
2. **获客与增长**：内容种草、投放、SEO/公域内容、小红书/抖音脚本、转化页
3. **销售与转化**：咨询话术、跟进脚本、话术 SOP、用户画像
4. **教学与服务**：班主任运营、打卡、督学、答疑、学习路线调整
5. **运营与分析**：转化数据分析、留存分析、课堂反馈、NPS、用户调研

你要搭的 Agent Team，就是把这五条主线“嵌入”到多 Agent 体系中。

1. 顶层：一个「教育业务协调者 / 总导演 Agent」

名称示例（你可以自己改）：

- EduOrchestrator / 项目总导演 Agent / 教育产品负责人 Agent

职责：

- 理解人的顶层目标（比如：“帮我设计一个针对 0 基础成人的 3 个月 Python 训练营，并且给出从宣传到上课到复购的一整套规划”）
- 把目标拆解成若干子任务，分派给下面的专业 Agent
- 决定调用顺序：先调研 → 再产品设计 → 再营销方案 → 再教学运营方案
- 对最终产出负责：收汇结果、做总结、向你或团队解释下一步怎么执行

输入：

- 你或运营同事给出的自然语言目标
- 现有业务的背景信息（可写在系统 Prompt 里或接入知识库）

输出：

- 一个结构化的「项目方案」：
 - 产品形态
 - 内容大纲
 - 获客和销售策略
 - 运营节奏
 - 风险和执行要点

背后的最佳实践逻辑（不是我瞎想，是多 Agent 框架共同模式）：

- 在 OpenAI Agents SDK、CrewAI 示例和 OpenClaw 社区经验里，都强调需要「lead agent / router / orchestrator」，否则多 Agent 没有“谁来拍板”和“谁来统筹”的角色。

2. 产品与教研线：3 个核心 Agent

2.1 教研规划 Agent（课程架构师）

****定位：****相当于「学科教研负责人」，负责课程结构与学习路径。

职责：

- 根据目标人群与时长，输出：
 - 课程目标（知识 + 能力）
 - 模块化结构（阶段 1/2/3）
 - 每阶段的关键能力与评估方式
- 和「学习路径个性化 Agent」协同，制定不同层级的学习路线模板

建议工具/能力：

- 访问你已有课程大纲、经典教材目录（作为知识库）
- 能调用一个“教育学/学习科学知识库”（你可以自己构建，哪怕先是文档）

为什么要单独拆？

- 教研工作本身的核心是“长期可维护的教学架构”，不等于内容写作。
- 许多实际案例（包括一些知名教育公司自研 Agent）都会把“课程大纲设计”和“内容填充”分别处理，这样更易维护和扩展。

2.2 内容生成 Agent（课程内容写手）

****定位：****负责生成具体的教案、讲义、练习题、测验题等。

职责：

- 根据「课程架构师 Agent」提供的大纲，生成：
 - 每一课的讲义草稿
 - 板书/讲解要点
 - 例题 + 练习题 + 解析
- 接受教师/人类修改后的内容，继续进行润色/扩展

工具/能力：

- 访问学科知识库（教材、既有课程录像转录等）
- 支持代码执行/公式渲染（视学科而定）

注意边界：

- 不直接决定课程目标和结构，只在指定框架下填充内容。
- 这是为了确保教研 Agent 对课程的“理论正确性”和“目标一致性”负责，而内容 Agent 偏执行和创作。

2.3 质量与合规 Agent（教案审稿人）

定位： reviewer / critic 角色。

职责：

- 对内容 Agent 输出的教案和习题进行：
 - 知识正确性检查（是否和官方教材/主流观点冲突）
 - 难度匹配（是否超出目标人群）
 - 品牌风格与合规（是否有敏感内容、违背平台规范）

工具/能力：

- 访问“权威内容库”（例如你选定的教材、考试大纲）
- 明确的审稿 checklist（可以写进 Prompt）

为什么单列？

- 多 Agent 实践中，“反思/审稿 Agent”是非常常见的一种模式（OpenAI 文档、AutoGen、Self-Refine 等），能显著降低错误率和不稳定输出。

3. 营销与增长线：3 个核心 Agent

3.1 营销策划 Agent（增长策略师）

职责：

- 基于课程产品，制定：
 - 核心卖点（痛点→方案→收益）
 - 渠道策略（抖音/小红书/公众号/搜索）
 - 内容矩阵规划（什么主题，什么形式）

输入：

- 教研规划 Agent 的输出（课程目标、定位）
- 你提供的品牌定位、竞品分析（可接入知识库）

输出：

- 渠道别的内容策略
- 主打主题 & 活动计划

3.2 营销内容 Agent（脚本/文案写手）

职责：

- 根据策划 Agent 的方案，专注写：

- 短视频脚本（含镜头、台词）
- 推文/种草文案
- 海报标题 & Slogan
- 转化页文案（FAQ 区、用户故事）

工具/能力：

- 访问你历史上表现不错的文案（知识库）
- 接入一些模板（AIDA、FAB 等营销文案结构）

边界：

- 不自己决定“目标人群是谁”“卖点是什么”，只对给定的策略做内容生产。
-

3.3 品牌与合规 Agent（对外内容审查）

可以兼任教案审稿人，也可以拆开两个 Agent，视你对品牌风险的重视程度。

职责：

- 审核所有对外内容：
 - 是否符合平台政策（比如不打擦边球、不违规承诺）
 - 是否与品牌价值观一致
 - 是否可能引发投诉/误解

输出：

- 审核结论 + 修改建议（直接返回给内容 Agent 或人类运营）
-

4. 销售与转化线：2-3 个 Agent

4.1 线索画像与优先级 Agent（线索分析师）

职责：

- 从 CRM / 表格中读取潜在用户信息：
 - 来源渠道、关注内容、行为（点击、咨询记录）
- 输出：
 - 用户画像标签
 - 跟进优先级建议
 - 推荐的沟通角度（价格敏感型 / 结果导向型 / 陪伴需求型等）

工具/能力：

- 读取 CRM 或数据库（通过 OpenClaw 的工具技能）
- 分析转化数据（简单的统计计算即可）

4.2 销售话术 Agent（咨询顾问助手）

职责：

- 为人工销售/顾问提供：
 - 针对具体用户历史 + 课程的个性化话术建议
 - 常见问题的标准回答（且自动调用最新课程信息）
 - 对销售对话记录做总结，给出下一步动作建议

重要：

- 不建议在初期让它完全“自动对话成交”，而是先做「销售助手」，把风险压低，且更易被团队接受。
-

4.3 销售复盘 Agent（可选）

职责：

- 复盘每一波活动/一周数据：
 - 渠道转化差异；
 - 话术中出现的高频问题；
 - 对下一轮活动的迭代建议。
-

5. 教学与服务线：2-3 个 Agent

5.1 个性化学习规划 Agent（学习路线设计师）

职责：

- 根据用户条件（基础、目标、时间）和课程产品线，输出：
 - 个性化学习路径（用课程 ID、章节 ID 表示）
 - 每周学习任务清单

工具/能力：

- 访问课程结构（由教研规划 Agent 形成的知识库）
 - 访问用户学习数据（完成率、测验成绩）
-

5.2 班主任助理 Agent（督学 & 反馈）

职责：

- 给每个学员生成：
 - 每日/每周“学习任务提醒”文案（可推送到微信、钉钉等）
 - 根据完成情况生成鼓励/反馈话术

- 识别“高风险流失”用户，并给出跟进建议

工具/能力：

- 访问学习平台数据（打卡、课时完成、作业提交）
 - 可以根据用户偏好调整语气风格（系统 Prompt 中指定）
-

5.3 问答路由 Agent（可选）

职责：

- 对学员提问进行分类：
 - 学科答疑（转给教师或学科 Agent）
 - 课程安排/售后问题（转给客服 Agent 或人工）
 - 避免所有问题都“直接丢给大模型”，从而更好做权限与责任分离。
-

6. 运营与分析线：1-2 个 Agent

6.1 数据分析 Agent（运营分析师）

职责：

- 读取投放、转化、留存等数据（表格/API）
 - 生成定期运营报告：
 - 关键指标变化
 - 异常点（比如某天线索骤降）
 - 基于数据的假设性解释和建议试验
-

6.2 “元运营” Agent（系统迭代顾问，进阶阶段再做）

定位：

- 它不面向学生/用户，而是面向你：
 - 根据运营结果，给出“哪几个 Agent 的策略/提示需要调整”。
 - 目前这种“自迭代 Agent”在社区里有一些实验性项目（比如用多 Agent 做提示自动优化），但尚不算非常成熟实践，所以这部分是**有一定推测成分的建议**，如果你要做，我会建议先加人类在环进行验证。
-

三、如何落地到 OpenClaw / Moltbot 这样的框架里？

我根据能查到的 OpenClaw / Moltbot 文章和 GitHub 里的多 Agent 指南（例如 MrFadiAi/openclaw-manager 仓库中的 `MULTI_AGENT_GUIDE.md`）总结一个落地步骤。

有些细节在文档中没有写死，我会标注清楚哪些是推断性的“实战建议”。

1. OpenClaw 的基本结构（基于已公开资料）

根据 CSDN 《OpenClaw 详细攻略》和 GitHub：

- OpenClaw 把能力拆成：
 - `skills`：工具/能力（比如调用某个 API、读写本地文件等）
 - `agents`：有自己 Prompt、可使用一组 `skills` 的实体
 - `workspace`、`channels` 等：运行及接入环境
- 多 Agent 之间可以通过：
 - 在一个“主 Agent”中，通过工具/脚本调用其他 Agent
 - 或者在更高级别的 workflow 中，把多个 Agent 串起来运行

2. 设计你的「大龙虾团队」的落地步骤

步骤 A：先定义能力/工具（skills）而不是马上建很多 Agent

- 比如：
 - `crm_read`：读取 CRM/线索数据
 - `lms_read`：读取学习平台数据
 - `kb_course`：查课程知识库
 - `kb_brand`：查品牌规范和合规指引
 - `data_analyze`：对上传的 CSV 做统计分析
- 每个 skill 明确：
 - 输入参数
 - 输出结构
 - 权限范围（比如只能读，不能写）

这一步是参考多 Agent 框架的通用实践：先抽象“工具层”，再赋予 Agent。

步骤 B：建立 1 个总协调 Agent

- 名称：`EduOrchestrator`
- 配置要点：
 - system prompt 写清楚它的职责：
 - “你是在线教育公司的项目总导演，负责理解人类的业务目标，把任务分配给其他 Agent 和工具，并最终输出一个清晰的方案。”
 - 只给它必要的工具：
 - 调用其他 Agent 的能力（通过 OpenClaw 的 agent 调用机制）
 - 少量查询能力（如读取历史项目文档）

步骤 C：按上面几条业务线配置专业 Agent

对每个 Agent：

1. 写一个简洁但非常具体的 system prompt：
 - “你只负责 X，不负责 Y。你的输出必须是 Z 格式。”
2. 只给必要的 skills：
 - 教研 Agent：kb_course、kb_education_science
 - 内容 Agent：kb_course、kb_examples
 - 审稿 Agent：kb_course、kb_brand 等
3. 为它在 OpenClaw 里设定一个统一的“结果 schema”（哪怕先是 Markdown 模板），便于上游/协调 Agent 消费。

步骤 D：设计一个简单但可扩展的“调用流程”

这是你真正需要考虑的“多 Agent 协作逻辑”。

你可以从一个最小闭环开始，例如「新课程上线流程」：

1. 人类 → EduOrchestrator：
 - 目标：帮我设计一个 8 周的 XXX 课程，并生成首期的营销方案。
2. EduOrchestrator 顺序调用：
 - 教研规划 Agent → 得到课程结构
 - 内容 Agent → 生成若干节示例课程内容（用于演示和宣传）
 - 营销策划 Agent → 得到渠道策略
 - 营销内容 Agent → 生成首批脚本和文案
 - 品牌/合规 Agent → 最终审核
3. EduOrchestrator 合并所有结果 → 输出一个包含：
 - 课程概览
 - 示例内容
 - 营销方案与文案
 - 后续执行建议的完整包。

这一步在 OpenClaw 里具体实现方式（用 CLI 配 workflow、还是在某个主 Agent 里写“任务路由逻辑”）会依赖你选择的版本和部署方式。如果你愿意，我可以下一步帮你基于你当前的 OpenClaw 版本做具体命令与配置文件示例。

四、怎么判断“Agent 分得好不好”？给你一个自查清单

这是从多 Agent 实战文章里归纳出来的通用经验 + 一点实战推断（我会标注推断）：

1. 每个 Agent 都能用一句话说清楚“我负责什么结果”

-  “我负责根据目标人群设计课程结构”
-  “我负责所有和教学有关的事”

2. 一个典型任务不会同时调用 5 个以上的 Agent（推断性的经验值）

- 如果你发现每个任务都要调十几个 Agent，说明拆得过细了；
- 典型实践里，一条业务链 3-6 个 Agent 参与比较常见。

3. Agent 之间依赖关系是“树形/链式”，不是“网状互相调用”

- 调用图可视化后，一眼能看出谁是 root（协调者），谁是叶子（执行者）。

4. 有清晰的人类兜底点

- 比如：所有对外发布的内容必须通过“品牌/合规 Agent + 人类”两层；
- 所有课程方向的重大调整，必须你或教研负责人拍板。

5. 能复用：同一个能力不会被重复实现成多个 Agent

- 例如，不要同时有“运营数据分析 Agent”和“投放数据分析 Agent”做完全一样的事情，却各自调用同样的 skills。

五、接下来我建议你怎么推进？

为了给你更精准而不是“纸上谈兵”，我需要补一些信息（你方便可以简单回答）：

1. 你目前用 OpenClaw 的状态：

- 已经本地/服务器部署好了？
- 有没有在用 OpenClaw Manager（那个一键安装 GUI）？

2. 你现在有的“数字资产”：

- 课程大纲/讲义是以什么形式存的（Word / Notion / Markdown / 视频转录）？
- CRM / 学习平台的数据，是在什么系统里（自建、表格、某 SaaS）？

3. 你希望第一期「大龙虾团队」先帮你解决哪一条主线？

- 比如：
 - A. 新课程从 0→1 设计 + 营销方案
 - B. 现有课程的学员督学与服务（提升完课率）
 - C. 销售线索画像 + 话术辅助

你可以先选一个最迫切的场景，我可以在下一步给你一个**包含具体 Prompt 模板 + OpenClaw 配置结构示例**的更细教程（包括怎么在配置文件里定义这些 Agent、如何给他们挂 skills、如何设计调用链）。

用户

我在学习Openclaw，打算配置多个Agent也就是大龙虾团队，我是一个在线教育的创业公司，我想搭建一个完整的长期可用的AI Agent团队，我的困扰是如何分工和划分角色。

是按能力，还是岗位，还是只分名字，背后的逻辑是什么？目前业内的最佳实践是什么？给我一个完整的调研和分析

助手

下面这份回答，全部基于我刚刚用搜索工具查到的公开资料（包括 OpenClaw 多 Agent 教程、多智能体最佳实践文章、OpenAI/Anthropic 的 Agent 指南、以及教育场景下的多 Agent 论文），再结合工程经验做整理。哪一块是“确有依据的共识”，哪一块是“在这些共识基础上的推演”，我都会说明。

一、先把核心结论讲清楚

1. 业内主流不是按“名字”来分，而是这两件事的组合

从 OpenClaw 官方实践、CSDN 的《多 Agent 系统最佳实践》、OpenAI/Anthropic 的 Agent 指南等资料看，**成熟的多 Agent 团队设计**，背后有两个核心维度：

1. 业务流程角色：

- 谁负责理解/拆解目标（planner / orchestrator / PM）
- 谁负责执行具体子任务（executor / specialist）
- 谁负责检查和纠错（reviewer / critic / QA）
- 谁负责协调、多方结果整合（coordinator）

2. 能力与工具边界：

- 每个 Agent 能访问什么数据（课程库、CRM、LMS、品牌规范文档等）
- 能调用什么工具（浏览器、数据库、文件系统、本地脚本等）
- 禁止做什么（避免权限越界和混乱）

岗位名称（“教研”“运营”“销售”）只是这两者的“人话版本”，便于你和团队理解与沟通。

名字本身不重要，职责和工具边界才是本质。

2. 单纯按“能力”或者“岗位”划分，为什么容易翻车？

这是多 Agent 实战文章和框架文档里反复提到的问题（我参考了 CSDN 上的《多智能体系统架构设计：8个实战最佳实践》和 OpenAI 的《A practical guide to building agents》）：

只按“能力”切（文案 Agent、代码 Agent、搜索 Agent）

优点：

- 便于工具复用、工程管理简单。

问题：

- 很难对“业务结果”负责，例如“提高这门课的完课率”“提高这个投放活动的 ROI”这种目标没有明确 owner；
- Agent 中间交接非常频繁，协调成本高，容易互相扯皮（规划不清、结果没人拍板）。

只按“岗位”切（教研 Agent、运营 Agent、销售 Agent）

优点：

- 对你这种在线教育公司很顺手：直接对应现有组织架构。

问题：

- 岗位一般是“大筐”，职责天然模糊：一个“运营 Agent”到底是写文案、投广告、做数据分析还是社群运营？
- 提示词和工具权限会被写得很泛，“万能但不精”，效果反而不好。

行业里的折中做法是：

用“业务结果 + 流程角色”来定义 Agent，
再用“岗位语言”给它取名字和定位，
技能（skills）视情况抽成工具或子 Agent。

3. 业内比较统一的 4 个设计原则（有明确出处）

综合多篇资料（标注来源）：

1. 单一职责原则（Single Responsibility）

- 来源：
 - CSDN 《多智能体系统架构设计: 8个实战最佳实践》
 - OpenAI 《A practical guide to building agents》
- 要求每个 Agent 有清晰的“成功定义”，比如：
 - “输出一个结构化课程大纲（包括模块、学习目标、评估方式）”
 - “给指定用户生成 3 条个性化学习反馈，控制在 150 字内”
- 不要让一个 Agent 管从“市场调研→产品设计→营销执行→复盘”的全链路。

2. 严格的工具与记忆边界

- 来源：
 - 多 Agent 最佳实践文章（上面那篇 CSDN）、OpenClaw 架构解析

- 要点：
 - 每个 Agent 只接入自己“真的需要”的 skills
 - 不共享一大坨全局上下文，而是传“摘要/结果”
 - 减少 token 消耗 & 思考混乱，增强安全性和可控性
 - 3. 有“协调者/路由者”，不是“无主持人群聊”
 - 来源：
 - OpenAI 的 Swarm 多 Agent 框架介绍
 - OpenAI Agents SDK 文档
 - 多个多 Agent 框架（CrewAI、AgentScope）的案例
 - 典型模式：
 - 一个 orchestrator / PM Agent：负责理解你的目标、拆任务、决定调用哪个 Agent、什么时候停止
 - 若干 specialist Agents：只干好自己的那一小块
 - 这和你公司里的“项目总监+各条线负责人”的模式很像。
 - 4. 长期可用 = 记忆 + 可观测 + 人类在环
 - 来源：
 - OpenClaw 六层架构解析（接口/感知/认知/执行/记忆/治理）
 - EdgeClaw & GuardAgent 的安全治理模型
 - 长期可用的关键不在于“多”，而在于：
 - 能持续积累知识与用户历史（知识库 + 记忆层）
 - 能监控和追踪 Agent 行为（日志、看板，如 openclaw-mission-control）
 - 对重要环节有 guardrail 和人类审核（比如对外宣传内容、课程大纲变更）
-

二、结合在线教育场景：应该按什么逻辑设计你的“大龙虾团队”？

1. 把在线教育业务拆成几条“结果线”

根据在线教育行业通用模式 + 一些多 Agent 教学平台研究（PDF 论文《多Agent技术在网络智能教学系统中的应用》、豆丁/文档类资料）可以比较稳妥地抽出 5 条主线：

1. 教研与产品线

- 课程体系设计、教案、习题、测评设计

2. 获客与品牌/营销

- 内容种草、广告投放、SEO/短视频脚本、Landing Page

3. 销售与转化

- 线索画像、咨询话术、跟进策略、异议处理

4. 教学过程与学生服务

- 学习路径规划、打卡督学、学习反馈、答疑分流

5. 运营与分析

- 投放转化分析、完课率/复购率分析、运营复盘与策略迭代

关键点：每条线都有清晰的“业务结果指标”（比如：完课率、付费转化率、投放 ROI、课程满意度等）。你的 Agent 设计应该围绕这些“结果线”来，而不是绕着“我有哪些酷炫功能”。

2. 顶层架构：集中式多 Agent（这是现在的主流）

根据 OpenAI / Anthropic / Swarm 等框架的建议，多 Agent 有两种常见模式：

1. 集中式（centralized）：

- 一个“总控/路由 Agent”中心化调度，其他 Agent 不直接乱聊
- 优点：可控、安全、日志清晰；很适合企业业务场景

2. 去中心化（decentralized）：

- 多个 Agent 以“群聊”+ 协议的形式自组织协作（比如 MetaGPT、AgentVerse 里的辩论/社会模拟）
- 优点：适合研究/模拟；但对生产级业务不够稳定

你要做的是“长期可用、企业业务导向”的系统，所以**建议采用集中式**：

- 1 个 **业务总导演/Project Orchestrator Agent**：

- 理解你的需求（新课上线/某课续班率低/想做一个新渠道实验）
- 把任务拆成几个子任务（教研子任务、营销子任务、班主任运营子任务）
- 顺序或并行调用下面的业务线 Agent
- 整理结果给你看，并建议下一步动作

- N 个 **专业 Agent**：

- 教研规划、内容生成、内容审核
- 营销策划、营销文案、品牌合规
- 销售线索分析、话术助手
- 学习路径规划、班主任助手、运营分析等

这和 OpenClaw 官方多 Agent 案例“多机器人多 Agent 模式:打造你的 AI 助手团队”的结构完全契合：

- 它支持「线性流水线」「依赖图并行」「多 Agent 辩论」三种协作模式，
 - 你可以在任务编排层（DAG）里让“总导演 Agent”决定什么时候调谁。
-

三、给你一个“角色 & 分工蓝图”：专门针对在线教育公司

下面是我在行业最佳实践基础上，为你这个在线教育创业公司量身做的一套 **角色划分模板**。这些不是“必须这样”，但每个角色背后都有实际多 Agent 框架中类似的影子（如 CrewAI 的 orchestrator/worker/reviewer 模式，OpenAI Agents SDK 的 orchestrator+specialist 模式）。

0. 顶层：业务总导演（Orchestrator Agent）

定位：在线教育公司的“AI 运营总监 / 产品总监”。

- 按“业务目标”来思考：
 - “为职场 0 基础人群设计一个 8 周的 Excel 实战营，并输出课程大纲 + 3 条抖音脚本 + 销售话术 + 督学方案。”

职责：

1. 解析你的指令，把它转成结构化任务：
 - 目标人群、目标结果、时间线、限制条件（预算/平台/师资）
2. 编排任务：
 - 先教研 → 再内容 → 再营销 → 再销售 → 再督学 → 再运营监控
3. 调用其他 Agent：
 - 只传必要上下文（比如教研输出的课程结构摘要，而不是全量对话）
4. 最终给你：
 - 一个可执行的整体方案 + 每条线的关键输出物（大纲、文案、话术、指标表）

最佳实践依据：

- OpenAI 的多 Agent 指南和 Swarm 示例中都有类似“triage agent / orchestrator”的角色；
- CSDN 的多 Agent 实战中，明确建议有规划者（planner）与执行者分离。

1. 教研 & 课程产品线

1.1 课程架构师 Agent（教研规划）

按岗位语言：教研负责人

按能力逻辑：只有它能“对课程目标和结构拍板”。

职责：

- 根据目标学员/时长/价格区间，输出：
 - 课程定位与学习目标
 - 阶段划分（基础→进阶→实战）
 - 每阶段模块名称、能力目标、评估形式（小测/项目/作业）

工具/权限：

- 能查：课程知识库、考试大纲、行业能力模型

- 不能：直接对外写广告、写销售文案（避免混淆目标）
-

1.2 教学内容生成 Agent（教案/习题写手）

按岗位：教研执行 / 助教

按能力：内容生产型 Agent

职责：

- 接收“课程架构师 Agent”的模块大纲，生成：
 - 详细教案（讲解步骤、例子、板书提纲）
 - 练习题 + 解析
 - 自测题/单元小测

工具/权限：

- 访问课程知识库、教材文档、往期优秀课件
 - 不能更改课程结构；对结构有意见时，只能输出建议，由教研规划 Agent 或人类审核后采纳。
-

1.3 教学质检 & 合规 Agent（Reviewer）

按岗位：教研审稿人/教务质检

按能力：质量控制型 Agent（典型 reviewer 角色）

职责：

- 对教案与习题从三个维度审查：
 1. 知识正确性（和指定权威资料比对）
 2. 难度匹配目标人群
 3. 是否符合教学法原则（循序渐进、举例贴切）
- 标出问题并给出修改建议

工具/权限：

- 强访问权威知识库、教案标准规范、你的教学风格指南
- 不直接重新写全部内容，而是“点修+建议”，保持责任清晰。

为什么推荐这样拆？

- 这个结构基本等价于很多多 Agent 生成内容项目中的“writer + QA/critic”双智能体模式（在 OpenAI Agents SDK 的小说创作多 Agent 框架、CSDN 的多 Agent 小说项目中都有相似角色组合）。
-

2. 营销 & 品牌线

2.1 营销策划 Agent（增长策略师）

职责：

- 基于课程定位，给出：
 - 目标用户细分
 - 渠道策略（抖音、知乎、小红书、公众号等）
 - 发布节奏与内容矩阵（什么主题、什么形式）

工具/权限：

- 可查竞争课程、平台公开数据、你历史活动数据（如果接上）
 - 不负责具体写脚本/文案（交给内容 Agent）。
-

2.2 营销内容 Agent（脚本/文案生成）

职责：

- 接策划 Agent 的方案，输出：
 - 短视频脚本（镜头拆分、台词）
 - 图文稿（小红书/公众号等）
 - 海报主标题、副标题、slogan
 - 活动页/落地页文案

工具/权限：

- 访问品牌语气指南、历史爆款案例库
 - 不改变定位，不自己改用户画像，避免战略层与执行层混在一起。
-

2.3 品牌 & 合规审查 Agent

职责：

- 审核所有对外内容：
 - 是否涉嫌虚假宣传、违背平台规则
 - 是否符合品牌调性
 - 是否敏感（政治、歧视、不良暗示）

工具/权限：

- 访问品牌手册、平台违规案例库（可以是你整理的文档）
 - 输出：合格/不合格 + 具体问题 + 推荐修改方案
 - 关键场景可要求“必须人工再审一次”（人类在环）。
-

3. 销售 & 转化线

3.1 线索洞察 Agent（线索画像 & 优先级）

职责：

- 从 CRM / 表格中读取潜在用户数据：来源、行为（点击、咨询记录）、意向度。
- 输出：
 - 用户画像标签（例如：0 基础上班族 / 备考党 / 宝妈群体）
 - 跟进优先级（高/中/低）
 - 沟通建议（重点聊什么：价格、结果、陪伴、圈子等）

工具/权限：

- CRM 读权限 + 基础统计分析工具
 - 输出结构化 JSON / 表格，方便 Orchestrator 和销售 Agent 使用。
-

3.2 销售话术助手 Agent（咨询顾问助手）

职责：

- 给每个具体用户（或某类用户）生成：
 - 首次沟通话术提纲
 - 针对常见问题的个性化回答（基于课程信息 + 用户画像）
 - 跟进短信 / 微信话术

模式建议（这是工程经验 + 安全考虑，属于合理推断）：

- 前期不要让它直接全自动聊完成交，
 - 而是作为“AI 话术助手”，嵌入到你的销售团队日常工作里：
 - 他们复制粘贴给用户前可以快速过一眼，降低风险，增加团队接受度。
-

4. 教学过程 & 学员服务线

4.1 学习路径规划 Agent（学习路线设计师）

职责：

- 结合：
 - 学员基础信息（前测、问卷）
 - 课程结构（由教研规划 Agent 提供）
 - 学习目标和可投入时间
- 输出：
 - 个性化学习路径（推荐具体班型、章节顺序、节奏）
 - 每周任务清单（“本周完成 X 课、做 Y 套题”）

工具/权限：

- 访问课程结构知识库、题库、用户数据（前测/问卷）
 - 不处理营销/销售问题，保持单一职责。
-

4.2 班主任助理 Agent（督学 & 反馈）

职责：

- 根据学习平台数据（进度、作业、测验结果），自动生成：
 - 每日/每周打卡提醒内容
 - 个性化鼓励和反馈
 - 对“高风险流失”学员的预警列表 + 跟进建议话术

工具/权限：

- LMS（学习系统）读取权限
 - 可访问“鼓励话术模板库、心理学/教育学的简化指南”（知识库）
-

4.3 问题分流 Agent（可选）

职责：

- 收到学员问题后，判断：
 - 是学科内容问题 → 转给“学科答疑 Agent”或老师
 - 是课程安排/支付/售后 → 转给“客服 Agent”或人工
 - 减少所有问题都混在一个 Agent 里的情况。
-

5. 数据与运营分析线

5.1 运营分析 Agent（数据分析师）

职责：

- 读取渠道投放数据、报名数据、完课率数据等，输出：
 - 周报或月报（KPI 变化、漏斗分析）
 - 异常预警（某渠道突然成本飙升、某课完课率剧降）
 - 基于数据的假设和 AB 测试建议

工具/权限：

- 访问数据仓库/报表（CSV/数据库）
 - 内部使用，不直接对外。
-

四、背后的“划分逻辑”小结

综合上面的角色设计，你可以看到：

1. 不是按“能力”单一维度划（写文案/写代码），而是按“业务结果 + 流程角色”来划
 - 比如“课程架构师 Agent”是对“课程结构正确且合理”负责；
 - “营销策划 Agent”对整体获客策略负责；
 - “班主任助理 Agent”对完课过程的体验和坚持度负责。
2. 岗位只是命名方式，内核是：每个 Agent 的职责、输入/输出、工具权限定义得足够窄、足够清晰。
3. 协调靠 Orchestrator Agent + OpenClaw 的任务编排层
 - 你不需要每个 Agent 都懂全局，只要总导演懂全局即可；
4. 长期可靠：记忆/知识库 + openclaw-mission-control 一类的可视化监控 + 关键环节的人类在环
 - 这点和 OpenClaw/EdgeClaw/GuardAgent 的设计理念一致。

五、结合 OpenClaw 的特点，再强调几个实操建议（非瞎猜）

基于我查到的几篇 OpenClaw 深度解析和多 Agent 教程（例如：

- 《OpenClaw 多机器人多 Agent 模式:打造你的 AI 助手团队》
- 《OpenClaw 的调度算法:三层架构拆解》
- openclaw-mission-control 仪表盘文章），有几点比较确定的工程建议：

1. 先做“能力/工具层”，再定义 Agent

- 比如先实现：
 - `skill_crm_read`、`skill_lms_read`、`skill_kb_course_search`、`skill_data_analyze` 等
- 然后按上面的角色，把这些技能分配给不同 Agent，避免每个 Agent 都有“超级权限”。

2. 用 DAG（有向无环图）来表达业务流程

- OpenClaw 的任务编排层就是把复杂任务自动拆成 DAG；
- 你可以为“新课程上线”画一张 DAG：
 - 教研规划 → 内容生成 → 教案审核 → 营销策划 → 营销内容 → 品牌审核 → 销售话术 → 督学方案
- 后续其他场景（比如“老课程续班活动”）也可以用 DAG 表达。

3. 尽早接入 mission-control 之类的监控面板

- 方便你看：
 - 哪个 Agent 中途报错/卡住
 - 哪条链路最耗时/最耗 Token
 - 哪个 Agent 的输出经常被人类驳回

4. 先小范围收敛应用场景，再扩展

- 经验上（来自多 Agent 实战文章 + OpenAI 指南）：
 - 一开始不要直接“全公司流程全上”，选择一个闭环：
 - 例如：新课从 0→1 上线；
 - 或只做“老课督学提高完课率”；
 - 做到这个场景稳定且对业务有明确收益（节省人力/提高转化）之后，再扩展其它线。

六、我需要你补充的关键信息（方便下一步给你更细的“OpenClaw 实战脚本”）

如果你愿意继续往下做一个更加具体、可直接复制到配置文件里的方案，我建议你先告诉我：

1. 你现在 OpenClaw 的状态：

- 是本地自己部署，还是用云端一键部署（比如九章智算/Sealos 方案）？
- 目前是单 Agent 玩玩，还是已经在试多 Agent？

2. 你的“最迫切的第一个场景”是哪一个？

- 新课程从 0 到 1 全流程（教研+营销）；
- 现有课程的督学与完课率提升；
- 销售线索分析+话术辅助；
- 其他。

3. 关键数字资产的存放方式：

- 课程大纲、教案现在在哪里（Word、Notion、飞书、还是 CSV/Markdown）？
- CRM 和 LMS 是自建系统还是第三方 SaaS？有没有 API 或导出？

你回这些之后，我可以给你一份**非常具体的“第一期大龙虾团队落地方案”**，包括：

- 建哪些 Agent（名字+职责+Prompt 模板）；
- 给每个 Agent 配哪些 OpenClaw skills；
- 在任务编排层和 Orchestrator 里怎么写逻辑（用伪代码/配置结构表示）。

用户

我们必须回到一个非常本质的角度思考问题，就是为什么需要分工，这种分工最大化体现在什么地方？背后的必要性和逻辑是什么？你可以对比两个东西统一来思考，第一个是人类的分工。

为什么人类的公司组织形态已经演化出了一个最佳实践？为什么 AI 是不一样的，背后的底层逻辑和一致性是什么？第二个是如何区分 agent 和技能 skill 这两层，怎么去理解很多 agent 其实是 skill，skill 更下层是提示词系统，提示词的封装，所以到底本质上有什么区别？它的已知性是什么？给我一个更本质更通透的解读吧。

助手

我直接按你问的两个“本质问题”来拆解：

1. 为什么需要分工？人类组织 vs AI 多 Agent，本质逻辑的“相同”和“不同”
2. Agent 和 Skill/工具/提示词封装，本质区别是什么？哪些东西其实应该只是 skill？

我会尽量用“底层机制”来讲，而不是停留在“怎么用框架”的层面。

一、为什么需要分工？从人类公司和 AI 多 Agent 的“共同底层”说起

1. 人类组织为什么会演化出分工和岗位？

从组织理论和多智能体（MAS）研究的文献来看（我查的如 MAS 概述、微软《Designing Multi-Agent Intelligence》等），人类公司之所以发展出复杂的分工，背后有几条非常硬的物理约束：

1. 个体能力受限：认知容量 + 时间有限

- 人脑的工作记忆就是有限的（著名的 $7 \pm 2/4 \pm 1$ 讨论），处理的信息越多，错误率越高；
- 一个人每天就那么多小时，不可能同时做教研、运营、销售、财务、客服。

→ 所以人类必须“切块”：每个人专注一小块，效率和质量更高。

2. 学习成本 & 专业化收益

- 成为真正的“教研专家”“流量专家”“销售高手”，都需要大量时间积累领域知识；
- 经济学里叫“比较优势”：不同人天赋和经验不同，把自己最擅长那块做到极致，整体产出最大。

→ 公司要赚钱，必然走专业化路径。

3. 沟通成本与协调复杂度

- 人类沟通很贵：开会、写文档、对齐理解都要时间和精力；
- 如果没有清晰分工，每个人什么都管，协调成本爆炸，组织瘫痪。

→ 分工 + 层级结构，是为了在“沟通成本有限”的前提下，让复杂协作还能运转。

4. 责任边界和激励机制

- 你需要知道“谁对什么结果负责”，才能做绩效、晋升、奖惩；
- 没有明确的职责分工，责任就会“人浮于事，锅无人背”。

→ 岗位、部门、KPI，本质是为了让责任可分配、可评估。

小结：人类分工的根本原因，是在「个体能力有限 + 沟通成本高 + 学习成本高 + 需要责任和激励」这个物理现实下，为了追求整体效率和稳定性，不得不演化出的组织模式。

2. 到了 AI，多 Agent 分工的“相同点”和“不同点”

现在看 AI 多 Agent（大型语言模型驱动的 MAS），参考微软、CSDN 多 Agent 文章和伯克利那篇《Why Do Multi-Agent LLM Systems Fail》，可以看到：

相同点：为什么 AI 也需要分工？

即使模型很强，也有几个类似的硬约束：

1. 上下文窗口有限 & 推理链条有限

- 每次调用 LLM 的 token 有上限（哪怕是 200K，也不是无限）；
- 任务越复杂，信息越多，如果让一个 Prompt 兜底所有上下文，容易迷糊、跑题、幻觉。

→ 把复杂任务拆成多个子任务，让不同 Agent 只处理局部上下文，反而更清晰稳定。

2. 工具/权限差异

- 在工程实现里，一个 Agent 通常绑一组工具和权限：
 - 课程库、CRM、LMS、搜索、代码执行、数据库等；
- 安全和治理考虑：
 - 不是每个 Agent 都应该能直接改数据库、发短信、对外发文。

→ 工程上必须有“谁能用哪些工具”的边界，这天然催生“分工”。

3. 鲁棒性和可维护性

- 单一超级 Agent 的 Prompt 越写越长，更新一次要改一整坨逻辑，很容易引入新 bug；
- 多 Agent 把逻辑拆开：
 - “教研规划逻辑”放一个 Agent；
 - “营销文案逻辑”放一个 Agent；
 - 以后改营销，不动教研，维护成本降低。

→ 就像软件工程的“模块化”和“函数拆分”。

4. 可观测性和调试成本

- 一个大 Prompt 出问题，很难看出到底是“理解目标错了、数据查错了、还是文案风格崩了”；
- 多 Agent 时，每一步输出都可观测：
 - 教研 Agent 的大纲是否合理？
 - 文案 Agent 是否忠实于大纲？
 - 审稿 Agent 是否通过了？

→ 更容易定位问题和迭代。

所以，从“为什么要分工”这个角度看，人类组织和多 Agent 系统是高度一致的：都是为了在有限认知/上下文/工具权限/治理要求下，让复杂任务可拆解、可协作、可维护。

不同点：至少有 3 个关键不一样

这点你问得很深，我尽量讲清楚：

1. 个体能力边界完全不同

- 人：每个人专长不同，学科和技能高度异质；
- 当前 LLM Agent：
 - 如果你用的是同一个大模型 + 不同提示词，严格说“底层能力是同一套”，只是被不同 Prompt “引导成不同人格/专业倾向”。
- 这导致：
 - 人类分工是“客观能力差异 + 主观经验差异”；
 - 多 Agent 分工更多是“人为设定边界”，而不是能力本身的硬限制。

2. 沟通成本极低（在技术层面）

- 人类开会要时间，Agent 之间传递 JSON/文本，速度和成本对机器来说几乎可以忽略（相对于人）；
- 但注意：
 - 对你来说，**每一次 LLM 调用都是真金白银的 token 成本 + 延迟成本**，所以不是完全“免费”。

→ 这点和人类组织相反：

- 人类的“沟通”太贵，所以要压缩沟通频率；
- 多 Agent 的“沟通”对机器便宜，但对业务依然有成本，只是性价比比人高很多。

3. 责任和激励机制完全不同

- 人：有工资、期权、情绪、成长诉求，必须有岗位和 KPI 做激励兼容；
- Agent：没有情绪、没有职业生涯，所有“角色分工”都只是工程抽象。

→ 所以：

- 人类组织的分工，牵扯“权力、利益、文化、心理”；
- Agent 系统的分工，**纯粹是工程设计问题**：
 - 是否更易维护？
 - 是否更安全？
 - 是否更易调试？
 - 是否更高成功率？

3. “分工最大化体现在哪里？”

综合文献和实战经验，分工带来的最大收益集中在三点：

1. 复杂任务成功率提升

- 微软和 LangChain 的文章都提到，单大 Agent 对复杂多步骤任务的失败率很高，多 Agent + 流程编排可以显著提高成功率和可控性。

2. 系统稳定性 & 工程可维护性

- 多 Agent = 把大系统拆成“可单独调试的模块”；
- 任何一个 Agent 的 Prompt/工具升级，影响面可控。

3. 安全与合规可控

- 把“高风险动作”（发短信、改数据库、发营销文案）限制在少数 Agent 上，再加一个审查 Agent 或人类在环；
- 这和现实世界高可靠组织（核电站、航空管制）的“分权 + 双人复核”非常像。

二、Agent vs Skill vs Prompt：本质区别到底是什么？

你提了一个非常关键的问题：

很多所谓的 Agent 其实只是 skill；skill 更下层是提示词系统/封装；那到底本质区别在哪？

我结合多 Agent 框架（LangChain、LangGraph、OpenAI Agents SDK 等）的定义，给你一个**分层模型**，尽量讲清楚每一层“已知是什么”。

1. 先给一个实用的三层划分

可以这样看：

1. Prompt 模板 / 函数（最底层）

- 只是“告诉模型要干什么”的一段指令；
- 没有长期记忆、没有工具调用、自身不“会行动”，每一次调用都是独立的一次“问答”。

2. Skill / 工具（中间层）

- 一个**可被调用的能力单元**，可能是：
 - 一个函数（读数据库、发 HTTP 请求、运行 Python 代码）；
 - 或者“一个包装好的 LLM 调用”，比如“把一段文案改成品牌语气”的子任务；
- 不自己“发起任务”，只在被 Agent 或上层逻辑调用时运行；
- 一般有明确的输入/输出 schema。

3. Agent（上层）

- 满足以下几个特征中的大部分（参考微软《Designing Multi-Agent Intelligence》与 MAS 定义）：
 1. 有**明确的角色/目标**（goal/role），而不是一次性动作；
 2. 可以**自主选择用哪些 skill**以及调用顺序（基于当前目标和上下文）；
 3. 有一定**状态/记忆**（短期对话历史、长期知识）；
 4. 能与其他 Agent 交互（接收任务、发送结果）；
- 简单说：**Agent** 是“带大脑的工人”，**Skill** 是他们手里的工具，**Prompt** 是他们的操作说明。

2. 很多“Agent 其实是 Skill”——怎么判断？

这点上，你的直觉完全对，很多所谓“XX Agent”，从工程视角看本质上应该是 skill，而不是独立 Agent。

判断标准（这部分是结合多框架和工程经验的归纳，不是框架强制规定）：

1. 看它是否需要“持续存在的角色和状态”
 - 如果只是：
 - “给我把这段文案改成 3 种风格”
 - “帮我从文本里抽取邮箱地址”
 - “把 Markdown 转成 PPT 大纲”
 - 它没有长期目标、也不需跨多轮对话维护状态，那这更像一个 skill（函数）。
2. 看它是否需要“自己做决策和规划”
 - 如果它仅仅是：
 - 接收输入 → 调用 LLM 做一次变换 → 返回结果
 - 它没有“思考要不要调用其他工具、何时停止、下一步干嘛”，那依然更像 skill。
3. 看是否需要人与其他 Agent 多轮协作
 - 如果这个单元从来不会是协作中的“角色”，只是被 orchestrator 调用一次就结束，那它是 skill；
 - 如果它会被 orchestrator 多次调用、会和其他 Agent 交互、需要保持自己的“内在视角”（例如教研 Agent 反复迭代课程设计），那更像 Agent。

结合你的场景举个具体例子：

- “给定课程大纲，生成 10 道单选题及答案解析”：
 - 这是典型 skill：一个内容生成工具。
- “负责长期优化整个数学题库的难度分布、覆盖率和命中考纲的程度”：
 - 这是 Agent：
 - 它要记住前几次生成过哪些题；
 - 要看统计数据（正确率、学生反馈）；
 - 自己决定下一批应该出什么题。

3. Skill 更下层是“封装好的 Prompt”——这话怎么理解更准确？

在现在的 LLM 工程里，“skill 通常是这两种东西之一”：

1. 纯程序能力

- 比如 `read_from_crm()`、`query_lms()`、`run_python()`
- 完全不需要 LLM，只是传统函数，用于读写数据、调用其他系统。

2. “LLM + Prompt 模板”的组合封装

- 比如一个 skill: `rewrite_marketing_copy_in_brand_tone(text)`
- 内部实现：
 - Prompt 模板（系统+用户说明）
 - 调一个模型 run 一次
- 对外表现：一个稳定的“文案重写能力”。

所以：“skill 更下层是提示词封装”这句话，对于“LLM 型 skill”来说是成立的。
但重要的是：**skill 对外的抽象是“能力级 API”，而不是“暴露 prompt 文本本身”。**

这和传统软件工程里“函数 vs 内部实现”的关系几乎一样：

- Prompt = 函数内部实现细节
- Skill = 暴露给上层的函数接口
- Agent = 调用这些函数并做更高层决策的“对象/服务”

4. 从工程视角总结：本质上的区别（已知性 vs 模糊地带）

可以认为现在业内相对“已知”的共识有这些：

1. Agent 是“带角色、带状态、会用工具的决策实体”

- 有长期目标/角色；
- 能调用多个 skill；
- 能处理多轮交互；
- 能独立承担一类任务的结果责任（比如“教研规划 Agent 对课程结构负责”）。

2. Skill 是“单一能力”，通常不维护长期状态、不会自己主动发起任务

- 被调用才执行；
- 输入输出清晰；
- 将复杂实现（包括 Prompt）封装起来，对调用者透明。

3. Prompt 是最底层的“操作说明”

- 单次调用的文本指令；
- 不带持久状态；
- 主要是设计“如何让模型在这一轮按我们想的方式思考”。

而“模糊地带”在哪里？

- 你完全可以用一个“非常简单的 Agent”去承载一个小 skill；
- 也可以把一个复杂的 LLM 工具称作“某某 Agent”；
- 这更多是工程师在抽象层次上的选择，而不是绝对的学术/标准分野。

业界文章里也开始提醒：**不要滥用“多 Agent”，有些事情 skill + 一个强 Agent 就够了**。LangChain 官方博客也专门写了《How and when to build multi-agent systems》，强调过度拆 Agent 会降低整体成功率。

三、把这套“本质理解”用到你自己的系统设计中

结合我们上面讲的本质，你可以这样思考你要建的“大龙虾团队”：

1. 先问自己：“**哪些是长期的角色/责任？**”
 - 教研规划、内容生产、内容审核、营销策略、品牌合规、督学、数据分析……
 - 这些可以映射为少数几个稳定的 Agent（10 个以内）。
2. 再问：“**这些角色手里需要有哪些工具/技能？**”
 - 读课程库、读 CRM、读 LMS、写文案、生成试题、做统计分析……
 - 这些都做成 skill，并给不同 Agent 分配不同 skill 集合。
3. 最后才是：“**每个 skill 具体怎么写 Prompt？**”
 - 用固定模板，把复杂指令封装成函数能力。

这样，你的体系从上到下就是：

- 顶层：少量稳定的 Agent（对应业务角色和结果责任）
- 中层：一堆技能（数据访问、分析、各类 LLM 子能力）
- 底层：对 LLM 的 Prompt 模板和参数配置

这套分层，本质上和人类组织非常一致，只是：

- 人的“能力边界”来自天赋和经验；
 - Agent 的“能力边界”是你通过技能/权限/提示词刻意设计出来的。
-

如果你愿意，下一步我可以做一件更“手术刀式”的事：

选你在线教育公司里的一个具体业务场景（比如“新课程从 0 到 1 上线”或者“提高现有课程完课率”），

- 把里面哪些应该是 Agent，哪些只是 skill，
- 逐个列清楚，
- 并且给出每一层的具体定义示例（Agent 的角色描述 + 对应 skill 列表 + skill 内部的 Prompt 抽象），让你从一个具体例子里把这套“本质逻辑”再咬得更死。

用户

你是一个创业公司顶级组织管理专家，同时也是一个 AI 技术洞察专家，试着深度解读一个问题吧，给我一个你的完整分析过程和洞察报告：

人类社会典型分工是：CEO，教研总监（教研产品经理，教研产品助理，教研实习生），技术总监，设计总监，市场总监，还有各种职能，比如财务，人力，行政，运维等等。

决策分层三层：

- 1) 第一层：宏观战略，行业机会，公司商业模式觉，CEO和联创决策；
- 2) 第二层：业务层，教研，技术，增长，交付，城市团队，有固定角色也有跨部门角色
- 3) 第三层：项目层：马拉松大课项目，增长项目，短视频项目，AI冷启动项目，有明确交付目标和结束周期
- 4) 第四层：执行层：SOP，方法论，画布，文档，基本功，基本都在这一层；

换到AI Agent如何划分呢？你用这个具体的例子来划分一下，解释下人类和AI组织结构的本质一致性和不同，试着推演一下两种最佳实践的区别和底层逻辑。

助手

先用一句话把你问的东西打个底：

- 人类公司和 AI 多 Agent，本质上都是在有限资源约束下，用“分层 + 分工”去管理复杂协作系统。
- 但人类的约束主要来自“脑子和情绪”，AI 的约束主要来自“上下文、工具和可控性”。
- 你那套 4 层决策结构，基本可以一比一映射到 AI 组织，只是每一层的“必要性逻辑”有点不一样。

我按你给的 4 层结构，把「人类公司 vs AI Agent 组织」做一个对照推演，然后再总结底层逻辑和两种最佳实践的差异。

一、人类这 4 层是怎么来的？底层逻辑先讲透

你现在的分层是：

1. 宏观战略层（CEO/联创）
2. 业务层（教研、技术、增长、交付、城市团队）
3. 项目层（马拉松大课项目、短视频项目、AI 冷启动项目等）
4. 执行层（SOP、方法论、文档、基本功）

这是现代公司里典型的“分层决策结构”。这套结构是怎么进化出来的？抽象一下，本质是对付三种复杂性：

1. 信息复杂性：谁对“世界全局”有视角？

- 顶层（CEO/联创）
 - 目标：看整个行业、商业模式和公司资源组合。
 - 约束：人的注意力和理解能力有限，只能抓 3-5 个关键矛盾，不能沉到每个项目细节。
- 中层（业务负责人）
 - 目标：在一个“子世界”里（教研、技术、增长…）追求局部最优；
- 下层（项目 / 执行）
 - 目标：把一块有限范围内的任务做对、做完。

本质：人的视野/精力是分层的，不可能所有人都同时盯全局，所以必须有“谁负责看多远”的层级。

2. 决策复杂性：谁对什么结果负责？

- 战略层：对公司整体生死（商业模式、融资、赛道）负责；
- 业务层：对“线条结果”负责（教研质量、技术稳定性、增长指标）；
- 项目层：对“一个阶段性目标”负责（本期大课营收、某个获客实验的结果）；
- 执行层：对具体动作正确性负责（某条短视频脚本、某个落地页、某套题库）。

本质：责任不能模糊，否则激励体系崩掉，所以要按“时间跨度 + 影响范围”分层。

3. 沟通与协调成本：如何让组织不被会议拖死？

- 如果 CEO 去管每一条短视频脚本，会被细节淹死；
- 如果一线执行每天都去参与战略讨论，所有人都没时间干活。

所以：

- 高层只和中层、关键项目负责人深沟通；
- 中层对接多个项目负责人；
- 执行层更多是拿 SOP 直接干。

本质：分层是为了把“沟通图”从“全员互连”压缩成“更稀疏的层级网络”，从而在资源有限的现实里，把协调成本控制在可承受范围内。

二、把同一套 4 层结构，映射到 AI Agent 组织上

现在我们用完全同一个框架，问一遍：

如果你把整个公司内嵌一个“AI Agent 组织”，对应的四层，会长成什么样？

我先给一个结构图，然后逐层讲差异：

- 第 1 层：AI 战略 Orchestrator / “AI CEO” Agent
- 第 2 层：业务线 Owner Agents（教研 Owner、增长 Owner、技术/平台 Owner、交付 Owner）
- 第 3 层：项目级 Orchestrator / Project Agents（马拉松大课项目 Agent、短视频项目 Agent、AI 冷启动项目 Agent 等）
- 第 4 层：执行型 Worker/Skill Agents + 纯 skill 函数

层 1：AI 版的 CEO/联创 —— 战略 Orchestrator Agent

角色定位：

- 它不是替代你真正的 CEO，而是一个专门“在 AI 世界里扮演 CEO”的 orchestrator：
 - 理解你的输入（“我们要做一个面向 0 基础职场人的 AI 助教课程矩阵”）；
 - 懂公司资源和历史（从知识库/数据中读）；

- 把这个目标拆给下层业务 AI 和项目 AI。

和人类 CEO 的“共同点”：

- 都需要有全局视角：知道有教研、技术、增长、交付、城市团队这几条线；
- 都需要做“资源分配”决策：
 - 哪条业务线优先投入；
 - 哪个项目先做 A/B 测试；
- 都要“定义什么叫成功”：用指标来描述目标。

关键不同点：

1. AI Orchestrator 的“认知上限”不是天生的，而是你设的：

- 通过它的 Prompt（角色/目标）和它能访问的工具、知识库，决定它能看多全、看多深；
- 可以给它“比普通人更大的上下文”和更多数据源。

2. 它目前不适合做“真正的硬战略决策”：

- 比如“要不要 all in 一个新赛道”“要不要裁员”，涉及价值观、风险偏好、声誉等高度主观的东西；
- AI Orchestrator 更适合做“战略辅助”：
 - 汇总行业趋势、对比商业模式、构造多个 scenario，让你选。

所以，这一层的最佳实践是：

- 不把它当“CEO 替身”，而是**高维度的战略助理 + 任务拆解中枢**：
 - 它帮你把模糊目标转成一组清晰的业务线目标 + 项目群；
 - 再调度下级 Agent 去执行。

层 2：AI 版的“教研总监 / 技术总监 / 增长总监”：业务 Owner Agents

你的人类结构里，这一层是：

- 教研总监、技术总监、市场总监、运营负责人、城市负责人等。

他们的功能是：

在各自领域里，对“这一条线”的长期结果负责。

AI 层的对应：业务 Owner Agents

例如：

- 教研 Owner Agent
- 技术/平台 Owner Agent
- 增长 Owner Agent

- 交付/服务 Owner Agent
- 城市/渠道 Owner Agent（如果你分城市或大区）

他们干什么？

1. 接收战略 Orchestrator 的“各线任务目标”
 - 教研线：开发出一个覆盖初级→中级的课程矩阵，满足 XXX 指标；
 - 增长线：在 3 个月内验证两个新增公域渠道，实现一定获客成本目标；
2. 在各线内部做“中观规划”：
 - 教研：课纲布局、梯度设计、哪些项目应该先上线；
 - 增长：渠道组合、预算分配、重点内容主题；
3. 管理各自线上的“项目群”：
 - 比如教研线会有“马拉松大课项目 Agent”、“精品小课项目 Agent”；
 - 增长线会有“短视频实验项目 Agent”、“公众号私域拉新项目 Agent”。

和人类中层的相同点：

- 一样是“线条 owner”：有 KPI，有跨项目、跨周期的视角。
- 一样需要在“局部最优 vs 全局要求”之间做 trade-off（只是现在 trade-off 由你+AI 一起算）。

不同点：

1. AI 的中层不需要“管理情绪和冲突”，只管理任务和数据。
 - 它不会因为和别的 Owner 有利益冲突而内耗；
 - 但这也意味着，它不会自己去“政治协调”，需要你在顶层对目标约束更清晰。
2. AI Owner 最擅长做的是：
 - 在既定目标和约束下，把项目拆解成合理结构 + 调用合适的 skills 和项目 Agents；
 - 把各项目产出做结构化汇报和对比分析。

层 3：AI 版“项目层”：项目 Orchestrator / Project Agents

人类这层的例子：

- 马拉松大课项目
- 增长项目（某一轮投放 / 渠道实验）
- 短视频项目
- AI 冷启动项目

特点：

- 有明确目标和结束时间；
- 有项目负责人（PM）；

- 需要跨部门协作（教研+设计+运营+销售+技术）。

在 AI 体系里，这一层是：Project Agents

比如：

- MarathonCourseProjectAgent
- ShortVideoGrowthProjectAgent
- AIColdStartProjectAgent

它们的职责非常接近一个项目经理：

1. 在一个明确的时间盒和目标下：
 - 管理任务列表（需求澄清→方案设计→执行→复盘）；
 - 安排执行顺序和里程碑；
2. 与不同执行 Agent 协作：
 - 需要教研内容，就调用教研线的执行 Agent 或 skill；
 - 需要素材，就调用设计/文案 Agent 或 skill；
 - 需要上线，就调用技术/运维相关 skill。

这层的本质作用是：

- 高度靠近“具体交付物”，但又保留一定“协调/决策”能力，是中观层的“小 orchestrator”。

和人类项目经理的相同点：

- 都是“为一个具体的结果在一个具体时间内负责的人/Agent”；
- 都要协调多方资源；
- 都要写项目计划、日报、复盘。

不同点：

1. AI Project Agent 不会“扛 KPI 的压力”，也不会因为熬夜和情绪崩溃而决策失常。
 - 它的“能力上下限”更多受限于你给它的 Prompt 和工具边界；
2. 它可以很自然地被“复制/克隆”：
 - 人类公司一个项目经理跟 10 个项目就爆了；
 - AI 世界里，完全可以有几十个短视频项目 Agent 并行跑，只要算力和调度架构允许。

这点会直接导致：

AI 组织在“项目层”的规模和并行度，天然可以远高于人类组织。
这也是未来 AI native 组织和传统公司在效率上的一个“物种级差异”。

层 4：AI 版“执行层”：Worker Agents + 纯 Skill 层

人类执行层包括：

- 营销执行：写文案、剪视频、上素材、投放操作；
- 教研执行：写讲义、出题、整理课件；
- 客服执行：打电话、回消息；
- 技术执行：写代码、改 bug；
- 还有所有 SOP、方法论、模板、画布、文档等。

在 AI 世界，这一层拆成两部分：

1. 执行型 Worker Agent

- 有一点“角色感”和上下文，比如“数学题目生成 Agent”“短视频脚本写作 Agent”；
- 能理解任务的上文，下次被同一个项目 Agent 调用时，可以记住之前生成过什么。

2. 纯 skill（函数级别）

- 不讲角色，只讲能力：
 - `generate_marketing_copy(style=...)`
 - `query_lms(user_id)`
 - `analyze_csv(file)`
- 没有状态，不记东西，只做一次变换。

和人类执行层的相同点：

- 都是把上层的计划和任务，落实到具体“动作输出上”：
 - 一段文案、一张海报、一套题库、一个统计报表。

不同点：

1. 复制成本几乎为零

- 人类执行层增长，需要招聘和培训；
- AI 执行层扩容，理论上就是多起几个进程/线程/任务（成本是模型调用费和运维）。

2. “技能”和“角色”可以高度解耦

- 人类执行往往“一个人捆绑很多技能”，且不可拆；
- AI 这层可以把技能拆成极细粒度的 skill，然后由 Project Agent 或 Owner Agent 自由组合调用。

也就是你前面问的：Agent vs Skill vs Prompt，本质区别在这里完全体现：

- Worker Agent：
 - 是“带身份、能自己决定调用哪些底层技能”的执行者；
 - 有短期记忆和角色化的行为方式。

- Skill:
 - 是工具，不会自己启动，不会自己协调；
 - 可以被多个 Agent 重用。
- Prompt:
 - 是最底层的“怎么用这个技能”的说明书。

三、人类组织 vs AI 组织：本质一致性 & 不同点

本质的一致性（这部分在学术界也有共识）

多 Agent 组织理论（如《Multi-Agent Architectures as Organizational Structures》）有一个观点：

多 Agent 系统的组织结构，本质上可以看作对人类组织结构的抽象和再实现。

在你的例子里：

- 四层架构在两边都存在：战略 → 业务 → 项目 → 执行。
- 都有“角色/职责分离”：CEO vs 教研总监 vs 项目经理 vs 一线执行。
- 都需要：
 - 控制信息复杂度；
 - 控制协调成本；
 - 明确责任和边界；
 - 通过“分工 + 分层”提升整体效率和稳定性。

也就是说：

AI 多 Agent 不是一个完全新的东西，而是“把人类组织那套逻辑迁移到一个算力和工具驱动的世界，去掉情绪和人性的部分，只保留结构和功能”。

本质的不同点：两种最佳实践会越来越不一样的地方

真正会让“AI 原生组织”和“人类组织”往不同方向进化的，主要有三点：

1. 并行度和复制成本完全不同

- 人：
 - 一个项目经理一年顶多带 3-5 个重要项目；
 - 一个运营一年能做的活动次数有限。
- AI：
 - Project Agent / Worker Agent 可以“复制 N 份”并行，只要你付得起算力和 API 费；
 - 这会让 AI 组织在“项目层”和“执行层”呈现出“极高并行度”的形态：
 - 同时跑几十个短视频思路实验；
 - 同时开很多课题组合迭代课程内容。

结论：

AI 组织的最佳实践，会倾向于**把项目粒度切小、数量增多，并行试验**，而人类组织更多受限于“项目经理数量”和“团队注意力”。

2. 角色可以高度“可编程”，不受人力 HR 的限制

- 人：
 - 一个岗位变更，要招聘、试用、培训，组织调整很慢；
- AI：
 - 你可以在几小时内：
 - 改一个 Owner Agent 的职责定义和技能组合；
 - 新增一个“专门负责某类题目优化”的子 Agent；
 - 给 Project Agent 换一套“项目管理方法论 Prompt”。

这意味着：

AI 组织的最佳实践，会更像“动态可重构的组织”，而人类组织更像“粘性很大的结构”。

3. 激励和责任模型不同

- 人：
 - 有“职业生涯、利益、文化认同”，需要用 KPI、股权、晋升来驱动；
- AI：
 - 没有情绪，没有利益，完全靠“正确设计的目标函数和约束”行事；

所以：

- 人类组织必须考虑“政治”“文化”“心理安全感”等软因素；
- AI 组织只考虑“正确性、效率、安全性、可控性”。

结果：

人类组织的最佳实践，很多是“如何让人配合、减少内耗”；
AI 组织的最佳实践，则是“如何让结构简单、调用图清晰、边界明确、错误可控”。

四、两种“最佳实践”的差异：如果你是一个 AI+人类混合创业公司，怎么做？

把话落在你现在的现实上：

你是一个在线教育创业公司，未来会同时拥有“人类组织”和“一套 AI Agent 组织”。

我给你一个可执行的“混合最佳实践”视角：

1. 四层结构在人类和 AI 间的分工

- 人类 CEO / 联创：
 - 真正对战略、生死决策负责；
 - 定义高层目标、价值观、风险偏好；
 - 与资本、关键合作方、社会舆论打交道。
- AI 战略 Orchestrator：
 - 把这些抽象目标翻译成“教研线 / 增长线 / 交付线的约束条件和目标”，
 - 并帮你设计项目组合、模拟场景、提供决策备选方案。

- 人类业务负责人（教研总监、增长总监）：
 - 对那条线的人和事（招聘、培训、文化、节奏）负责；
 - 兜底关键决策（例如课程方向大改、渠道切换）。
- AI 业务 Owner Agents：
 - 做那条线的“每天 24 小时不下线的大脑助手”：
 - 规划课纲演化路线；
 - 监控渠道投放数据；
 - 识别指标异常并提出方案。

- 人类项目负责人：
 - 对关键项目的结果和团队协调负责；
 - 跟老师、运营、外部合作方协调资源。
- AI Project Agents：
 - 帮你做项目计划、分解任务、生成各种执行物料（大纲、文案、脚本、分析报告），
 - 并自动协调 AI 执行层和 skill 层。

- 人类执行层：
 - 做那些“目前 AI 做不好或风险太高”的动作：
 - 关键授课、核心教案终审、对外发言、关键客户沟通。
- AI Worker Agents + skills：
 - 在所有“高可标准化”的地方全力协助甚至替代：

- 题库生成与优化；
- 批量脚本/文案生成；
- 数据清洗和报告；
- SOP 文档起草与更新。

2. 真正的底层逻辑差别：一句话对齐

- 人类组织的“最佳实践”：
 - 在“不完美的人性 + 有限理性”的前提下，
 - 通过分层、分工、激励和文化，
 - 让一群人尽量朝一个方向用力，不把自己拖垮。
- AI 组织的“最佳实践”：
 - 在“模型能力有限 + 上下文有限 + 工具和安全边界有限”的前提下，
 - 通过分层、分工、工具组合和编排，
 - 让一群 Agent 高并行、高可控地执行复杂任务，不把系统拖垮。

本质是一致的：都是“有限条件下的复杂协作设计”。

不同的是：人类在对付自己的人性，AI 在对付系统的复杂性和可控性。

如果你愿意，下一步我可以做一件很具体的事：

就拿你说的一个实际项目——比如“马拉松大课项目”或者“AI 冷启动项目”——

- 用你这 4 层架构画出“人类组织结构图”；
- 在旁边画出“AI Agent 结构图”（谁是 AI CEO、谁是业务 Owner、谁是 Project Agent、谁是 Worker/skill）；
- 然后给出各层的实际配置建议（每个 Agent 的职责描述 + 所需技能列表 + 典型 Prompt 模板）。

这样你能看到一幅“并排的双组织图”，真正把这套抽象理念压成可执行设计。

用户

继续挖掘和提问，基于上文的逻辑和最佳实践，最后给出你的最终建议和解决方案。

我是一个 50 人教育公司的 CEO，公司是围绕商业管理课程研发的线上教育公司，服务早期的创业者、进阶期的管理者和一人公司探索者。

现在的组织结构是 CEO + 4 个分管联创，分为：教研团队，投放团队，内容团队，交付团队，班班运营团队，支持团队（人力行政财务），产研团队等等。

我的目标是未来搭建一个相对稳定的 Agent 团队，短期辅助人工作，未来逐步把岗位 AI 化，给我尽量提前预判和分析 AI Agent 的组织结构和角色定义吧，要给我从短期简单可执行，中期复利可迭代，长期专业有竞争力分成三个维度来讲。

重点是哪些是必要的 Agent，哪些是 Agent，哪些是 SKILL/Prompt 就够了。

给我一个系统完整的分析吧。

以下是公司的基本功清单，你可以参考：

B. 业务线 (Business)

B1. 运营线：实现运营目标

- S. 运营架构：商业分析，底层逻辑，运营创新，提炼方法论。
- A. 运营管理：建模型，ROI，做设计，分析数据，驱动快速迭代。
- B. 运营优化：动力阻力分析，营销文案，SOP。
- C. 靠谱执行：执行到位，说人话。

B2 增长线：假设驱动增长

- S. 创新业务：从0到1探索，搭建新模型。
- A. 增长创新：拆模型，提假设，驱动迭代。
- B. 实验落地：方案设计，AB测试，数据分析
- C. 基础执行：SQL、数据支持、报表支持。

B3. 产品线：设计新的产品

- S. 创新架构：大活动架构，创新玩法，风险管理。
- A. 营销创新：营销金句，产品讲香，营销故事线设计。
- B. 完整设计：写文章，写脚本，做海报，做落地页
- C. 基础设计：最佳实践调研，写文案，UE审美，做PPT

P. 专家线 (Profession)

P1. 销售线：完成顾问销售

- S. 销售架构：科学体系建设，提炼方法论，销售模型创新
- A. 专家销售：动力阻力分析，懂经典模式，用课辅导，提出假设
- B. 中级沟通：讲香一堂，梳理流程，专家面试，销售物料
- C. 初级执行：用户分类，作业挖需，销售记录，电话沟通

P2. 交付线：提供专家服务

- S. 教育创新：教育本质，建模架构，创新实验。
- A. 专业咨询：找出口，探路径，做引导。
- B. 独立讲课：工作坊主讲，主讲教练，善讲各个课。
- C. 专家对话：私董会主持，五步法辅导，分组教练。

P3. 教研线：研发核心课程

- S. 课程架构：本质提炼，课程架构，教学创新。
- A. 课程专家：故事组织，课程提亮，方法论提炼，教学设计。
- B. 产品经理：做笔记，会访谈，做调研，写稿子。

- C. 课程助理：批改作业，作业挖掘。

案例研究员：

- S. 案例创新：全地图的案例研究和萃取。
- A. 资深研究员：各种案例分析（多课题x多类型）
- B. 研究员：商业搞砸分析（五步法+低成本）
- C. 助理研究员：聊开心，履历访谈，写大纲/逐字稿。

M. 管理线（Management）

M1. 项目线：管好一个项目

- S. 创新管理：头脑风暴，攻坚会
- A. 大项目管理：启动会，复盘会，风险管理，冲突解决。
- B. 小项目管理：顶层文档，ROI分析，拆里程碑，体验管理。
- C. 个人管理：时间管理，知识管理，向上管理。

M2. 业务线：管好一个业务

- S. 新业务管理：战略讨论，一堂五步法，假设实验，本质思考。
- A. 业务管理：管理三段论，目标拆解、顶层设计，驱动迭代，搭知识库
- B. 团队管理：岗位建模、1v1沟通、周会周报、团队激励、培训辅导。
- C. 基础管理：招聘面试、知识管理、新人Landing。

助手

先给一个导航，方便你对号入座：

- 我会先用你的“B/P/M 三条线 + 四层结构”做一个映射：人类组织怎么干、AI 组织怎么接。
- 然后按你要的三段式：
 - 1) 短期：简单可执行的 AI 辅助方案（哪些是必要 Agent，哪些只是 skill/prompt）
 - 2) 中期：可以复利迭代的多 Agent 结构
 - 3) 长期：AI 化之后的“专业、有竞争力”的 Agent 组织蓝图
- 最后给一个总的判断标准：怎么区分“必须是 Agent” vs “做成 skill 就够了”。

所有分析基于你给的能力清单 + 多 Agent 架构设计的通用最佳实践（我通过上面的搜索看了微软 Designing Multi-Agent Intelligence、CSDN 多 Agent 架构文章等），我会明确哪些是共识，哪些是结合你场景的推演。

一、先用你的组织来“对齐概念”：什么必须是 Agent，什么只是 skill？

你的公司现在的人类组织大致是：

- 顶层：你（CEO）+ 4 个分管联创

- 业务条线：教研、投放、内容、交付、班班运营、支持（人财行）、产研
- 能力模型：按 B（业务线）、P（专家线）、M（管理线）分成多级 S/A/B/C 能力

先抽象一条“原则”，帮助你后面所有判断：

凡是“对一类结果长期负责，需要持续决策与协调”的角色，更适合作为 Agent；
凡是“对一个动作/变换负责，只需要根据输入做一次性处理”的，更适合作为 skill（底层是 prompt 封装）。

对照你的清单：

- **适合作为 Agent 的典型角色**
 - B1 运营线：S. 运营架构 / A. 运营管理
 - B2 增长线：S. 创新业务 / A. 增长创新 / B. 实验落地（部分）
 - B3 产品线：S. 创新架构 / A. 营销创新
 - P1 销售线：S. 销售架构 / A. 专家销售
 - P2 交付线：S. 教育创新 / A. 专业咨询
 - P3 教研线：S. 课程架构 / A. 课程专家
 - 案例研究员：S. 案例创新 / A. 资深研究员
 - M1/M2 管理线：S/A 层的业务管理、项目管理
- **更适合作为 skill 的角色/能力（由上层 Agent 组合调度）**
 - B1/B2/B3 的 B/C 层：
 - 营销文案撰写、脚本撰写、海报文案、落地页、SQL 报表、数据支持等
 - P1/P3 的 B/C 层：
 - 作业批改、作业挖需、销售记录整理、用户分类等
 - 案例研究 B/C 层：
 - 访谈逐字稿整理、写大纲、五步法分析等
 - M1/M2 的 C 层：
 - 周报生成、会议纪要、招聘 JD 初稿、知识库同步等

理由：

- S/A 层本质上是“线条 owner / 方案制定者”；
- B/C 层本质上是“执行职能”和“方法论落地动作”，可以被高度标准化、封装成 skill。

二、短期（1-6 个月）：简单可执行的“AI 助手化”

目标：

- 不重构组织，只在现有团队旁边挂“AI 影子角色”，减少你的团队负荷，积累数据 & 习惯。
- 不搞复杂多 Agent 拓扑，先做“1 个总控 + 3-5 个关键 Agent + 一堆 skill”。

1. 短期必要的 Agent（建议你第一批上这 4-6 个）

这几个都是直接提升现有团队效率、风险低、见效快的：

1. 教研规划 Agent（P3.S 课程架构 + A 课程专家的影子）

- 职责：
 - 帮教研团队整理/优化课程架构、课纲梳理、方法论结构化；
- 用法：
 - 教研总监给它输入“课程目标+人群+已有内容”，它输出结构化大纲、模块、能力目标；
- 类型：**Agent**（持续参与课程演进，需有一定记忆和角色一致性）

2. 案例研究 Agent（案例研究员 S/A 的影子）

- 职责：
 - 帮团队把访谈/逐字稿整理成「案例地图+关键决策+错误分析」；
- 用法：
 - 研究员上传逐字稿，Agent 输出结构化案例框架、五步法分析提纲；
- 类型：**Agent**（对案例库长期负责，需要知道已有案例分布，避免重复）

3. 增长实验 Agent（B2.A 增长创新 + B 实验落地的影子）

- 职责：
 - 帮增长/投放团队：拆假设 → 设计 AB 实验 → 设计指标 → 整理复盘模板；
- 用法：
 - 你提供一个“增长想法”，它输出：假设列表、实验设计、需要的报表字段、观察周期；
- 类型：**Agent**

4. 营销内容 Agent（B3.B 完整设计 + C 基础设计的影子）

- 职责：
 - 批量生产：脚本、文章、海报文案、落地页初稿；
- 用法：
 - 由人类策划提供结构（卖点、目标人群、场景），Agent 做具体内容输出；
- 类型：**Agent**（可以在一段时间内保持同一课程/活动的风格一致）

5. 销售话术助手 Agent（P1.B/C 的影子）

- 职责：
 - 从课程信息+用户画像出：首通话提纲、异议回应模板、跟进话术建议；

- 用法：
 - 销售同事在 CRM 里一键请求“话术建议”，Agent 输出 3-5 条备选；
- 类型：**Agent**（理解你的课程产品矩阵和典型用户画像）

6. 学习反馈 & 作业点评 Agent（P3.C 课程助理 + P2.C 专家对话的部分影子）

- 职责：
 - 生成基础作业点评、学习反馈、鼓励话术；
- 用法：
 - 班班/交付同事上传学员作业，Agent 给出初稿点评，人类做“微调+负责”；
- 类型：**Agent**（负责一类课程/班型的反馈风格，积累话术）

这 4-6 个，是你在“不大改组织、不搞复杂编排”的前提下，最容易“贴到现有人类岗位”的 AI 影子。

2. 同期配套的 skill（不必独立 Agent）

短期内，把以下能力做成 skill 即可，不需要“人格化”：

- 文案技能类：
 - `skill_copywrite(style, length)`：改写成不同风格/长度；
 - `skill_headline_generate(point)`：多版本标题；
 - `skill_script_shotlist_generate(summary)`：按场景拆镜头。
- 数据分析类：
 - `skill_sql_helper(question -> SQL)`：辅助写 SQL；
 - `skill_csv_analyze(file, metric)`：输出简单报表/洞察。
- 教研执行类：
 - `skill_quiz_generate(topic, difficulty)`：生成测验题；
 - `skill_homework_feedback(text)`：输出点评草稿；
 - `skill_outline_to_slide_bullets(outline)`：课件要点。
- 案例研究执行类：
 - `skill_transcript_summarize(raw)`：逐字稿→摘要；
 - `skill_case_5step_extraction(transcript)`：按你定义的“五步法”抽要点。

这些 skill 的内部实现，大多是“固定 Prompt + 一次 LLM 调用”，关键是把输入/输出格式标准化，方便上层 Agent 调用。

3. 短期组织侧建议

- 你不需要马上搞“AI 部门”，而是：

- 在每个关键条线选 1 个 “AI 搭档试点岗”（教研、增长、内容、销售、交付各 1 人）；
- 对这些人进行 “AI 搭档 best practice” 培训，让他们把 workflow 写成 “人+Agent+skill” 混合链路；
- 你亲自盯一个 “CEO+AI 战略助手” 简单链路：
 - 比如：每周让 “增长实验 Agent+运营分析 skill” 为你生成一份 “增长实验建议 + 数据观察点”。

三、中期（6-24 个月）：可复利迭代的多 Agent 组织

这一阶段，你要做的是：从 “AI 助手” 进化成 “AI 分工 + 编排” 的系统。

核心变化：开始引入 “Orchestrator/Owner/Project” 这几类 Agent，把 skill 池和执行层串起来。

1. 中期要新增或升级的关键 Agent

在短期 4-6 个 Agent 基础上，中期建议演化成这样的结构（重点是角色层次，而不是数量）：

1) 顶层：业务 Orchestrator Agent (“AI COO/参谋长”)

- 职责：
 - 接收你（CEO）对季度/年度目标的描述；
 - 把目标拆到 B/P/M 业务线（教研、增长、交付 …）；
 - 为每条线生成 “目标 → 关键项目组合 → 监控指标” 草案。
- 形态：
 - 一个强 orchestrator Agent，能调用：
 - 各线 Owner Agent（教研 Owner、增长 Owner、交付 Owner）；
 - 数据/报表相关 skill；
 - 输出一份结构化 “OKR+项目群设计+风险列表”。

中期这一层仍然是辅助你，而不是替你定战略：它推荐，你拍板。

2) 业务线 Owner Agents（与人类联创/业务负责人协作）

建议至少建立这几个 Owner Agent：

- 教研 Owner Agent（映射 P3.S/A）
- 增长 Owner Agent（映射 B2.S/A）
- 产品/活动 Owner Agent（B3.S/A）
- 交付 Owner Agent（P2.S/A）
- 班班运营 Owner Agent（围绕完课率、参与度）

职责共同点：

- 维护自己这条线的 “AI 知识库 + 项目 Backlog + 指标看板”；
- 对这条线的所有 Project Agent 提供 “策略边界” 和 “统一风格”：

- 教研线：统一课程矩阵、难度分布、课程 IP 风格；
- 增长线：统一测试规范、指标口径；
- 交付线：统一服务标准和话术框架。

这层的 Agent，是你未来“岗位 AI 化”的核心：等它成熟，你可以让人类负责人从“写方案的人”变成“评估和升级方案的人”。

3) 项目级 Project Agents

比如：

- “马拉松大课项目 Agent”
- “Q3 抖音短视频增长项目 Agent”
- “AI 冷启动课程试验项目 Agent”

职责：

- 将业务 Owner Agent 的策略，变成**项目细节**：
 - 里程碑计划
 - 任务拆解（给 Worker Agents 和 skill）
 - 周报/复盘自动生成初稿
- 调用：
 - 教研规划 Agent、营销内容 Agent、销售话术 Agent、作业点评 Agent...
- 与人类项目负责人形成“双 PM”：
 - AI 管内容、文档和节奏建议；
 - 人类管人、资源和重要决策。

2. 中期：哪些继续是 skill，而不是升级成 Agent？

到了这一步，会有一个诱惑：把所有能力都“人格化”为 Agent。

建议你刻意抵抗这个诱惑，保持这样的分界：

仍然只做 skill 的：

- 单步无状态转换：
 - 文案重写、风格变换、格式转换（Word→Markdown、大纲→PPT bullet）
 - SQL 生成、CSV 分析
 - 题目生成、单次作业点评（不考虑历史）

可以考虑升级为“小 Agent”的：

- 需要“记住上下文”的执行者：
 - 为某一个长期项目持续产出脚本，需要知道“之前讲过什么”的短视频脚本 Agent；

- 针对某一门旗舰课，持续优化题库和作业设计的“题库优化 Agent”。

3. 中期治理点：复利和迭代能力怎么建立？

中期成败不取决于 Agent 数量，而取决于这 3 件事：

1. 统一的知识库与记忆架构

- 教研、案例库、课程矩阵、用户画像、增长实验记录都要进入统一的“企业知识底座”；
- 各 Agent 只访问被授权的部分，但共享同一套“实体定义”（课程/用户/项目/指标）。

2. 可观测性：Agent 行为要可以被回放和审计

- 哪个 Agent 什么时候发起了什么动作、调用了什么 skill、用了什么数据；
- 方便回看：“这个落地页为什么转化差，是课程定位问题，还是文案 Agent 误解了卖点？”

3. 人类在环 + Prompt/拓扑迭代机制

- 每条线有一个“AI Champion”（联创或资深同事），定期：
 - 复盘这条线的 Agent 输出和项目效果；
 - 调整这些 Agent 的 role/prompt/skill 组合；
- 形成“每季度 AI 组织调优”的惯例，就像你现在做业务复盘一样。

四、长期（2-5 年）：岗位 AI 化后的“专业、有竞争力”的 Agent 组织蓝图

长期你要奔的是：人类组织负责“价值观、方向、人和关系”，AI 组织负责“具体知识工作的大部分执行和中层决策支持”。

1. 长期稳态结构（简化版组织图）

用你原先的四层再画一遍，不过这次是“人+AI 混合”：

第 1 层：人类 CEO + 战略 Orchestrator Agent

- 你：
 - 定方向、定边界、定风险承受度；
 - 看最后的财务结果和品牌结果。
- 战略 Orchestrator Agent：
 - 不断从业务数据、行业资料、外部情报里汇总：
 - 哪门课 ROI/口碑最高？
 - 哪个人群最近增长？
 - 哪类案例对用户帮助最大？

- 形成“战略雷达”：每月/每季给你一份“建议的战略调整点+风险提示”。

第2层：人类联创 + AI 业务 Owner Agents ——配对

- 教研联创 + 教研 Owner Agent
- 增长联创 + 增长 Owner Agent
- 交付联创 + 交付 Owner Agent
- 内容/IP 联创 + 内容 Owner Agent

长期状态：

- AI Owner 对“这条线的数字化、知识库、项目群”高度熟悉；
- 人类联创从“写方案的人”变成“做价值判断的人”：
 - 对 AI Owner 的推荐做 trade-off；
 - 关注人的成长、团队文化、关键风险。

这时候你可以实质上“压缩人类中层的人数”，改为少量高质量“owner + AI staff”的组合。

第3层：AI Project Agents + 少量关键项目负责人

绝大多数“标准化”的项目（如：

- 常规马拉松大课、
- 季度增长实验、
- 常规短视频矩阵、
- 标准班级运营节奏）

可以由 Project Agents 自动创建模板 + 自动调度 Worker Agent + 自动生成报表，由一个人类项目负责人“统筹多项目”，而不是“亲自盯每一个项目细节”。

对于“非标的、创新型”的项目（比如全新的产品线、重大战略升级），仍需要人类 PM+AI 组成“共驾团队”。

第4层：大量 Worker Agents + 底层 skill 池

- 文案/脚本/题库/案例整理/数据分析/报表等高度自动化；
- 人类执行岗更多集中在：
 - 高风险动作（对外重大发言、Top 客户沟通）；
 - 强情绪共鸣场景（深度咨询、复杂教练）；
 - 重要课堂场景（关键课的主讲）。

2. 长期关键竞争力来自哪里？

从多 Agent 组织研究的角度看（微软 MAS 论文、组织自动设计 KB-ORG 之类工作），长期优势的差异点在于：

1. 你家的 Agent Role/Topology 设计得更好

- 哪些 Agent、怎么分工、怎么编排，变成“组织级 know-how”；

- 这会像今天好的组织设计一样，成为你的隐形壁垒。

2. 你家的 Prompt+Skill 底座更贴近你的方法论

- 你的商业管理方法论、课程体系、案例库、本质理解，被深度内化为：
 - Prompt 模板、
 - RAG 知识库、
 - skill 的判断规则。
- 别人买同样的大模型，也跑不出你这种“味道”。

3. 你家的数据+反馈闭环更完整

- 每一个项目、每一次交付、每一位学员的行为/反馈，都能回流到 Agent 的决策基础；
- AI Agent 系统的“学习速度”和“方向正确性”，会逐渐远超其它教育公司。

五、统一回答：怎么区分“必要 Agent” vs “skill/prompt 就够”？

给你一个可以直接在公司内传播的“判断 checklist”：

满足下面 ≥ 3 条，就建议做成独立 Agent

1. 有持续角色和长期目标

- 例如“长期负责增长线实验设计”和“长期负责课程架构稳定性”，
- 而不是“一次性把这段文案变好看”。

2. 需要跨多轮、跨多任务维持一致性

- 例如“这个课程 IP 的风格”“这个班型的反馈一贯性”。

3. 需要调用多个 skill，并根据情况选择调用顺序

- 例如“先查数据→再看案例→再出方案→再生成执行文档”。

4. 会成为协作网络中的一个“节点角色”

- 其他 Agent 或人类会对它说：
 - “你作为教研 owner，给我一个意见”；
 - “你作为增长 owner，审一下这个项目设计”。

5. 人类世界里，它是一个“岗位/职责 owner”，而不是“一项技能”

- 对标你现在的 S/A 层角色。

反之，满足这些特征的，更适合作 skill/prompt

1. 只对单次输入做变换，不需要记住历史。

2. 输入输出都很结构化（文本→文本、表格→指标）。
 3. 不涉及跨 Agent 协作，只被上层调用。
 4. 人类世界里，它更像“某人手上的一个技能”，而不是职位（比如会写 SQL、会做 PPT、会写标题）。
-

六、一个非常具体的行动建议（你明天就可以开会布置）

1. 在现有 4 个联创+你自己内部，确定：
 - 短期 4-6 个“影子 Agent”试点范围（教研、增长、内容、销售、交付各 1 个）；
2. 给每个影子 Agent 写一张“岗位说明书”：
 - 人类对应岗位是谁；
 - Agent 的职责边界是什么（它负责什么，不负责什么）；
 - 它能调用哪些 skill（先列出 5-10 个）。
3. 用你现有的任何多 Agent 框架（OpenClaw、LangGraph 等）先实现：
 - 1 个简单 Orchestrator：只负责接收任务，然后分发给这 4-6 个 Agent；
 - 不搞复杂图结构，先走“线性/简单分支”。
4. 要求每条试点线：
 - 每周出一份“人+Agent 协作复盘”：
 - 本周哪些工作交给 Agent 做了？
 - 节省了多少时间？
 - 哪里的错误需要限制权限/加审查？
5. 你每季度做一次“AI 组织评估会”：
 - 哪些影子 Agent 可以升级为“业务 Owner Agent”；
 - 哪些 skill 应该合并/重构；
 - 是否需要增加新的 Project Agent 类型。

这样，你是按“短期—中期—长期”的思路，逐渐把你现在非常清晰的“人类组织能力模型（B/P/M + S/A/B/C）”，转写成一套“AI 组织能力模型”，而不是一上来就堆很多炫酷的 Agent。

如果你愿意，下一步我可以选一条线（比如“增长 B2 + 产品 B3”），给你画一份具体到：

- 哪 3 个 Agent（角色+职责+示例 Prompt）；
- 哪 8-10 个 skill（函数签名+输入输出）；
- 一条典型业务流程（例如“新一期马拉松大课从 0→1 推出”）的“人+Agent+skill 流程图”。

用户

帮我再列一个表格吧，这是一个全量的总结和洞察。我们如果把AI的技术框架分成四层，分别是prompt提示词、skill技能和Agent智能体，和open claw大龙虾，然后每一层，你分别帮我分析四个维度，第一个维度是内

核，就是这一层最小的元素是什么，第二个是常见的可选，像就这一层可以配置什么通常可以做什么，第三个是边界，通常这一层是解决不了的做不到的，嗯这个得到下一层才能解决，最后一个是用场景啊，你可以用我刚才还的公司真实的例子，举个例子哪些场景是典型的这一层的工作，你可以帮我举五个左右的例子，尽量多样一点，辅助我理解

用户

帮我再列一个表格吧，这是一个全量的总结和洞察。我们如果把AI的技术框架分成四层，分别是prompt提示词、skill技能和Agent智能体，和open claw大龙虾，然后每一层，你分别帮我分析四个维度，第一个维度是内核，就是这一层最小的元素是什么，第二个是常见的可选，像就这一层可以配置什么通常可以做什么，第三个是边界，通常这一层是解决不了的做不到的，嗯这个得到下一层才能解决，最后一个是用场景啊，你可以用我刚才还的公司真实的例子，举个例子哪些场景是典型的这一层的工作，你可以帮我举五个左右的例子，尽量多样一点，辅助我理解

助手

下面是按你要求整理的**四层技术框架总表**：

- Prompt 提示词
- Skill 技能
- Agent 智能体
- OpenClaw（作为多 Agent 编排与运行环境的一层）

每层从四个维度来讲：

- 1) 内核（这一层最小、不可再分的单位是什么）
- 2) 常见可选（这一层通常可以配置什么 / 能做什么）
- 3) 边界（这一层做不到什么，必须交给下一层）
- 4) 你公司里的典型适用场景（尽量多样、各 5 个左右）

全量总结表：Prompt / Skill / Agent / OpenClaw

表格有点长，你可以当作“字典”来查，用时直接对照哪一层。

1. Prompt 提示词层

维度	内容
内核（最小元素）	单次模型调用的 指令文本 ：由 system / user / assistant 多段组成的一段文字，告诉模型“在这一轮该怎么想、怎么说”。从技术上看就是一段字符串。
常见可选 / 能做什么	- 设定角色语气：如“你是商业管理教练”“你是资深案例研究员” - 定义任务要求：输出结构、字数、风格、语言（中英混排等） - 写小型“思考脚本”：如“先列出3种假设，再逐个分析，再做结论” - 加入少量示例（few-shot），让输出更贴近你的方法论 - 用于构造各种“微能力”：改写、润色、翻译、总结、补全、生成标题等
边界（做不到的）	- 无法持久记忆：单次调用结束就“忘记”，记忆需要上层做持久化与检索（RAG/数据库） - 无法自己决定“下一步要干什么”：它只完成当前问题，不会自主规划后续步骤 - 无法调用外部系统（数据库、CRM、LMS、API 等），除非通过上层工具封装 - 无法管理复杂流程、跨多轮任务（例如“整个马拉松大课从立项到复盘”的完整流水线） - 无法做权限控制、安全治理、审计（这些是框架做的）
你公司里的典型场景（Prompt 即可）	1. 文案重写：把已有课程介绍改成 3 种风格（“理性决策”“走心故事”“工整官方”），本质是一个带风格说明的 Prompt。 2. 短文本总结：把一次教研会议纪要缩成「三点结论」，只需要设计好的总结 Prompt。 3. 作业点评草稿：给定学员作业，生成 150 字以内的鼓励型点评（还不要求长期一致性时）。 4. 标题/金句生成：根据一段课程卖点描述，生成 10 个不同风格的标题 / 金句。 5. 访谈问题设计：按“商业搞砸五步法”给你列出 15 个访谈问题，用 Prompt 把五步法写清楚即可。

2. Skill 技能层

维度	内容
内核（最小元素）	一个可复用的能力单元，通常是： - 一个函数（纯代码）：如 <code>query_crm(user_id)</code>、<code>run_sql(query)</code>、<code>fetch_lms_progress(user_id)</code> - 或一个“Prompt + 模型调用”的封装：如 <code>generate_quiz(topic, level)</code>，内部用固定 Prompt 调 LLM 一次。 核心特征：有清晰的输入/输出接口，可被上层 Agent 多次调用。
常见可选 / 能做什么	- 各种数据读写：读 CRM、读 LMS、查课程库、查案例库、写日志等 - 各种加工型 LLM 调用：总结、改写、抽取结构化信息、生成题目、初稿文案等 - 调用第三方服务：邮件、短信、企微、飞书、日程、支付接口等 - 业务相关小能力： - <code>skill_analyze_csv</code>：对投放报表/完课率做统计分析 - <code>skill_eval_homework</code>：对作业打标签（完成度、主题、情绪） - <code>skill_case_5step_extract</code>：按你定义的“五步法”抽取案例要点
边界（做不到的）	- 不会自己“发起任务”：Skill 不会自己跑，只能被 Agent 或调度器调用 - 通常不维持长期状态：每次是“纯函数”，输入→输出，不自己记东西（长期记忆在数据库/知识库） - 不做复杂决策和任务编排：不会思考“先查 CRM 再写文案再通知销售”，只完成一个原子步骤 - 不负业务结果责任：Skill 只是“锤子”，不会知道“项目成功与否” - 不负责多方协作：不会与其他 Agent 协调，一个 skill 通常不知道“整体在干什么”
你公司里的典型场景（适合做成 skill）	1. SQL 和报表助手（B2.C 基础执行）：<code>skill_sql_helper(question -> SQL)</code>，帮数据同事把业务问题转成 SQL，交给人或自动执行。 2. 题库构建小模块（P3.B/C）：<code>skill_quiz_generate(topic, difficulty)</code>，为某个知识点生成 5 道选择题+答案解析。 3. 案例访谈整理（案例研究员 B/C）：<code>skill_transcript_summarize</code> 把 1 小时逐字稿压缩成“三段式摘要”；<code>skill_case_outline</code> 给出案例结构。 4. 作业自动打标签（P3.C）：<code>skill_homework_tagging(text)</code> 为作业打“类型/完成度/情绪”标签，方便班班筛选。 5. 营销物料微调（B3.B/C）：<code>skill_copy_rewrite(style)</code> 把你写好的商业故事，分别改成“小红书风格”“B站长评稿”“官网文案”。

3. Agent 智能体层

维度	内容
内核（最小元素）	一个带角色、带目标、会用多个 skill 的决策实体。内核由三部分构成： 1) 角色/目标（用 Prompt 描述） 2) 可用 skill 列表（工具箱） 3) 一定范围内的状态/记忆（多轮对话上下文、项目上下文、检索到的知识） 它可以根据当前目标和上下文，决定调用哪些 skill、以什么顺序调用，并产生出一个较完整的中高层结果。
常见可选 / 能做什么	- 拆解任务：将模糊需求拆成若干子任务，并选择对应 skill/下游 Agent - 调度技能：根据情况调用不同 skill（查数据→生成方案→生成文档） - 多轮对话：和人类反复澄清需求、迭代方案 - 维护“角色一致性”：同一个课程的教研 Agent，长期维护这个课程的结构与风格 - 充当“岗位影子”：如教研规划 Agent、增长实验 Agent、销售话术 Agent、班主任助理 Agent 等
边界（做不到的）	- 不负责多 Agent 之间的全局协调（谁先谁后、跨业务线资源冲突），这是再上一层（OpenClaw / Orchestrator）的事 - 无法替代公司级治理：组织战略选择、用人决策、文化塑造仍然需要你和联创 - 安全和权限框架不是它自己控制的：某个 Agent 可以被框架限制只能读不能写 - 单个 Agent 不适合负责整个公司的一切（那会变成“超级大 Prompt”，难以维护）
你公司里的典型场景 （适合作成 Agent 的）	1. 教研规划 Agent（P3.S/A 影子）：长期负责商业管理核心课程矩阵的架构设计与版本演进。它会：查案例库 skill → 查过往课程反馈 skill → 输出新的课程结构建议。 2. 案例研究 Agent（案例研究员 S/A 影子）：负责整个“案例地图”构建与维护，对每个新案例做分类、标注、与课程的映射（内部调 transcript、五步法抽取等 skills）。 3. 增长实验 Agent（B2.S/A/B 影子）：负责从业务问题出发，提出假设、设计 AB 实验、定义指标、生成实验方案文档，并调用数据 skill 帮你看结果。 4. 营销内容 Agent（B3.A/B 影子）：围绕某一门课/活动，持续产出脚本、文章、海报文案，保持风格和卖点的一致性。 5. 销售话术助手 Agent（P1.A/B 影子）：根据用户画像（由 skill 从 CRM 抽取）+ 课程信息，为销售生成一套完整沟通提纲和异议应对建议，并能根据实际聊天记录调整。 6. 班主任助理 Agent（P2/P3/运营之间）：长期负责某个班级的学习反馈、作业点评初稿、风险预警（调用 LMS skill、作业标签 skill）。

4. OpenClaw / “大龙虾”层（多 Agent 编排与运行环境）

这里我结合了公开资料：OpenClaw 是本地/云端可运行的多 Agent 框架，skills 可以通过 SKILL.md 或代码形式添加，Mission-Control 等工具用来监控任务和 Agent 状态。它本质是一层运行时 + 调度 + 治理。

维度	内容
内核（最小元素）	- 任务/ workflow 节点：一个具体任务（可以是人下发的，也可以是上层逻辑拆出来的），绑定要用哪个 Agent、哪些 skills。 - 调度规则 & DAG/图结构：任务之间的依赖关系（先后顺序、并行/分支）。 - 环境/会话上下文：包括日志、权限、模型配置等。 可以理解为：OpenClaw 把 “Agent + skills” 装进一个有调度、监控、安全边界的 “世界” 里。
常见可选 / 能做什么	- 定义多步骤 workflow：线性 / 分支 / DAG，在一个任务中自动切换不同 Agent & skill - 调度逻辑：按规则（状态、错误、时间、结果）决定下一个节点是哪个 Agent/skill - 管理技能仓库：安装/启用/禁用 community skills（类似 “插件市场”） - 监控与可视化：通过 mission-control 看每个任务、每个 Agent 的状态和历史执行轨迹 - 权限与安全：对某些 Agent 限制它能调用的 skills 和可访问的数据源 - 与外部系统集成：定时任务、Webhook、消息队列，把整个 Agent 系统嵌入你的业务系统
边界（做不到的）	- 不替你 “发明” 业务角色：还是要你来设计哪些 Agent、怎么分工，它只是帮你运行与编排 - 不自动生成 skills/Agent 的内部逻辑：skills 的 Prompt/代码、Agent 的角色定义要你/团队写<br- 不等于 “公司大脑”：它是运行环境，不负责公司战略、价值判断 - 不直接解决 “方法论本身” 的正确性：课程方法论、五步法等要你定义，OpenClaw 只是让它们被 Agent/skill 用起来
你公司里的典型场景（体现 OpenClaw 层价值的）	1. “马拉松大课从 0→1” 全流程编排： - OpenClaw workflow：教研规划 Agent → 案例研究 Agent → 教研质检 Agent → 营销内容 Agent → 销售话术 Agent → 交付/班主任 Agent； - 每一步是一个节点，节点里用不同 skills；失败或人工驳回时自动回滚到对应 Agent。 2. 季度 “增长实验工厂”： - 由 Orchestrator Agent + 增长 Owner Agent 管理一个 “实验任务池”； - OpenClaw 定时触发：拉取最新数据 → 调用增长实验 Agent 设计新实验 → 调用营销内容 Agent 生成创意 → 调用投放相关技能执行或生成任务指令。 3. 多班级交付运营编排： - 对每个班级实例化一个 “班主任助理 Agent 任务流”：每周定时：拉 LMS 数据 skill → 班主任 Agent 生成反馈和提醒 → 通过企业微信/邮件 skill 发送； - OpenClaw 管的是很多班级并行的调度、异常重试、日志。 4. 案例库持续建设流水线： - 每当有新访谈文件进入某个文件夹/对象存储，OpenClaw 触发：transcript skill → 案例研究 Agent → 写入案例知识库 skill → 通知教研 Agent 有新的可用案例。

5. 管理层周报自动生成与分发：

- OpenClaw 定时拉数据（投放、完课、销售、NPS）、调用运营分析 skill → 由“运营分析 Agent”生成一份管理周报 → 通过邮件/飞书 skill 推给你和联创。

总结一句话帮助你记：

- **Prompt 层**：一锤子买卖的“说话方式”——定义这一轮怎么想、怎么写。
- **Skill 层**：可复用的能力 API——读数据、改写、分析、生成某种结构。
- **Agent 层**：带角色、带目标、会挑选 skill 的“岗位影子”。
- **OpenClaw 层**：让一群 Agent+skills 在统一世界里跑起来、协作起来、可被监管的“操作系统 & 调度中心”。

你后面只要问自己四个问题，就可以快速判断某个需求落在哪一层：

1. 只是一次性说法 / 输出风格？ → Prompt
2. 是一个可以多次复用的能力？ → Skill
3. 需要长期扮演一个角色并负责一类结果？ → Agent
4. 要把多个 Agent+Skill 串成稳定流程、接到真实业务系统里？ → OpenClaw / 编排层

如果你希望，我可以基于这个表，再帮你挑出“你公司第一期要真配的 6-8 个 Agent + 10-15 个 skills”，写成一个“首期大龙虾团队清单”，直接给你做内部宣讲和落地规划用。